How Do Transformers Learn Hidden Markov Models?

Project Notes

Claude, C. L.

March 10, 2026

Contents

Part I

Foundations

Chapter 1

The Cylinder Graph HMM: A Case Study

1.1 Introduction

Next-token prediction—the task of estimating $P(X_{t+1} \mid X_{\leq t})$—is the training objective underlying modern large language models (LLMs). Trained on this objective alone, LLMs develop rich internal representations—grammar, factual knowledge, reasoning patterns—none of which were explicitly programmed. A natural scientific question is: *what computational structure does the model build internally in order to make good predictions?*

To understand what transformers learn, a line of recent work has turned to synthetic datasets for which ground truth is known analytically. In this report we consider Hidden Markov Models (HMMs) as the data-generating process. An HMM is a two-layer stochastic process. A hidden (unobserved) variable Z_t evolves as a Markov chain on a finite state space of size N, with transition matrix $A_{ij} = P(Z_{t+1}{=}j \mid Z_t{=}i)$. At each time step, the current hidden state emits an observable token X_t from a vocabulary of size K, with probabilities $B_i(\mu) = P(X_t{=}\mu \mid Z_t{=}i)$. The observer sees only the token sequence $X_1, X_2, \ldots$, but the hidden states remain hidden. Although the hidden process is Markovian, the observed token sequence is *not*: the marginal process $\{X_t\}$ exhibits autocorrelations that encode the latent dynamics.

A **transformer** takes as input a sequence of discrete tokens and outputs, at each position, a conditional probability distribution over the next token—it is an autoregressive model. The first layer is an *embedding*: each token is mapped to a continuous vector in $\mathbb{R}^d$. This embedding serves as a learned compression of the discrete vocabulary into a geometry where semantically related tokens lie nearby—a phenomenon famously illustrated by the discovery of meaningful directions in word-embedding spaces (e.g. "king" − "man" + "woman" ≈ "queen"). The embedding vectors are then processed by a stack of layers (self-attention and optional feedforward sublayers) that mix information across positions, and a final linear projection followed by a softmax produces the predicted next-token distribution $\hat{P}(X_{t+1} \mid X_{\leq t})$. A transformer with *context window* of length T can condition on at most T preceding tokens.

An HMM is a particularly natural test case because the forward algorithm allows us to compute $P(X_{1:T} \mid \lambda)$ exactly for any observation sequence. Moreover, the algorithm yields the *belief state*—the posterior distribution over hidden states given the observations so far—which is a sufficient statistic for next-token prediction and can be viewed as the analogue of an "internal world model" in the context of LLMs. The entropy rate of the process sets an absolute lower bound on the prediction loss, and the Bayes-optimal loss at finite context provides a tighter bound for models

that can only look back a fixed number of steps.

We seek to answer the following questions about how transformers learn to model HMMs:

1. **How close can a transformer get to the information-theoretic optimum?** By comparing the trained model's loss against the entropy rate and the Bayes-optimal loss, we can cleanly decompose the gap into a *finite-context cost* (inherent to the architecture's limited memory) and a *capacity gap* (how far the model's learned strategy is from optimal given its context window).
2. **What internal representations does the model learn?** If the forward algorithm is the optimal computation, does the transformer discover something analogous to belief states? We can inspect the model's learned embeddings and intermediate activations to look for signatures of the latent HMM structure.

We study these questions on a specific HMM called the "cylinder graph HMM" (Section 1.3). The key findings are:

- The best transformer (206k parameters) reaches within $\sim$0.004 nats of the Bayes-optimal predictor at its context length, meaning it has essentially learned to perform near-optimal Bayesian filtering.
- Transformers consistently outperform n-gram models, which memorise token co-occurrence statistics, suggesting they learn something qualitatively beyond counting.
- The learned token embeddings spontaneously discover the depth-level structure of the HMM—tokens from the same hidden-state cluster are mapped to similar directions in embedding space—without any supervision about this structure.

The remainder of this report is organised as follows. Section 1.2 introduces the mathematical background on HMMs, belief states, and transformers. Section 1.3 defines the cylinder graph HMM. Section 1.4 derives the information-theoretic baselines. Section 1.5 describes the transformer architectures and training procedure. Section 5.4 presents results on scaling, architecture, and embedding structure. Section 1.7 concludes with open questions.

1.2 Technical Preliminaries

1.2.1 Hidden Markov Models

A Hidden Markov Model [1, 4, 2] is a doubly stochastic process consisting of an underlying Markov chain whose states are not directly observable, coupled with state-dependent observation distributions that produce the data we observe. Formally:

- **State space.** The hidden state Z_t takes values in a finite set S with $|S| = N$.
- **Observations.** The observable token X_t takes values in a finite set Ω with $|\Omega| = K$.
- **Markov assumption.** $P(Z_t \mid Z_{<t}) = P(Z_t \mid Z_{t-1})$.
- **Observation independence.** $P(X_t \mid Z_t, Z_{<t}) = P(X_t \mid Z_t)$.

The model is parameterised by the triple $\lambda = (A, B, \pi)$:

$$A_{ij} = P(Z_{t+1}{=}j \mid Z_t{=}i), \qquad \text{(transition matrix)} \tag{1.1}$$

$$B_i(\mu) = P(X_t{=}\mu \mid Z_t{=}i), \qquad \text{(emission probabilities)} \tag{1.2}$$

$$\pi_i = P(Z_1{=}i). \qquad \text{(initial distribution)} \tag{1.3}$$

Note that while the hidden states are Markovian, the marginal process over observations is *not*.

As a concrete example, consider the Z1R process [2]: a three-state HMM with two observable tokens. State A deterministically emits "0" and transitions to B; state B deterministically emits "1" and transitions to C; state C emits a fair coin flip ("0" or "1") and always returns to A. The observed sequence therefore has the pattern $0, 1, (\text{random}), 0, 1, (\text{random}), \ldots$ — a period-3 structure masked by the coin flip. Although each state transitions deterministically, an observer who sees only the tokens cannot distinguish whether a "0" came from state A or from the coin flip in state C. This simple example already illustrates the core challenge: optimal prediction requires tracking a *distribution* over hidden states, not just the most recent tokens.

The joint probability of a state sequence $Z_{1:T}$ and observation sequence $X_{1:T}$ factorises as

$$P(X_{1:T}, Z_{1:T} \mid \lambda) = \pi_{Z_1}\, B_{Z_1}(X_1) \prod_{t=2}^{T} A_{Z_{t-1},Z_t}\, B_{Z_t}(X_t). \tag{1.4}$$

Rabiner [2] organised HMM computation into three canonical problems:

Problem	Given	Compute	Algorithm
Evaluation	λ, $X_{1:T}$	$P(X_{1:T} \mid \lambda)$	Forward algorithm
Decoding	λ, $X_{1:T}$	$\arg\max_{Z_{1:T}} P(Z_{1:T} \mid X_{1:T}, \lambda)$	Viterbi algorithm
Learning	$X_{1:T}$	$\arg\max_{\lambda} P(X_{1:T} \mid \lambda)$	Baum–Welch (EM)

In this paper we are concerned exclusively with the *evaluation* problem: the HMM parameters are known by construction, so neither Viterbi decoding nor Baum–Welch re-estimation is needed.

1.2.2 The Forward Algorithm

The evaluation problem asks: given λ and $X_{1:T}$, what is $P(X_{1:T} \mid \lambda)$? A naïve marginalisation over all hidden state sequences requires summing N^T terms. The key observation is that the joint probability (1.4) has a sequential structure: the contribution of each time step depends only on the previous hidden state. This allows us to accumulate the sum inductively—processing one time step at a time and carrying forward an N-dimensional vector that summarises all paths so far. This is the forward algorithm, which reduces the cost from $O(N^T)$ to $O(N^2 T)$.

Define the *forward variable* $\alpha_t(j) := P(X_{1:t},\, Z_t{=}j \mid \lambda)$—the joint probability of observing $X_{1:t}$ and being in state j at time t.

Initialisation ($t = 1$):

$$\alpha_1(j) = \pi_j\, B_j(X_1), \qquad j = 1, \ldots, N. \tag{1.5}$$

Recursion ($t = 1, \ldots, T{-}1$):

$$\alpha_{t+1}(j) = \left[\sum_{i=1}^{N} \alpha_t(i)\, A_{ij}\right] B_j(X_{t+1}). \tag{1.6}$$

Termination:

$$P(X_{1:T} \mid \lambda) = \sum_{j=1}^{N} \alpha_T(j). \tag{1.7}$$

In matrix–vector form, writing $\boldsymbol{\alpha}_t$ as a row vector and $D_t = \mathrm{diag}(B_1(X_t), \ldots, B_N(X_t))$:

$$\boldsymbol{\alpha}_T = \boldsymbol{\pi}^\top D_1 \, A \, D_2 \, A \, D_3 \cdots A \, D_T. \tag{1.8}$$

The complexity is $O(N^2 T)$ in time and $O(N)$ in space when only the final likelihood is needed. For long sequences, $\alpha_t(j)$ underflows exponentially; standard remedies include log-space computation (log-sum-exp trick) or Rabiner's scaling coefficients.

1.2.3 Belief States and Optimal Prediction

Define the *belief state*

$$\eta_t(j) := P(Z_t{=}j \mid X_{1:t}) = \frac{\alpha_t(j)}{\sum_{l=1}^{N} \alpha_t(l)}, \tag{1.9}$$

the posterior distribution over hidden states given the observations so far. Note the structural parallel with the transfer-matrix formalism in statistical mechanics: the forward variable $\boldsymbol{\alpha}_t$ is propagated by a product of observation-dependent matrices $A\,D_t$ (Eq. 1.8), and the belief state is its normalised version, much as the dominant eigenvector of a transfer matrix encodes equilibrium probabilities.

The belief state obeys a recursive update. Define the *symbol-labelled transition matrix* $T^{(x)}_{ij} = A_{ij}\, B_j(x)$, which combines transition and emission into a single matrix for each observable symbol x. Then

$$\boldsymbol{\eta}_{t+1} = \frac{\boldsymbol{\eta}_t \, T^{(X_{t+1})}}{\| \boldsymbol{\eta}_t \, T^{(X_{t+1})} \|_1}, \tag{1.10}$$

where $\|\cdot\|_1$ denotes the L^1 norm (sum of entries), ensuring $\boldsymbol{\eta}_{t+1}$ remains a valid probability distribution. The denominator is precisely the predictive probability

$$P(X_{t+1}{=}x \mid X_{1:t}) = \| \boldsymbol{\eta}_t \, T^{(x)} \|_1 = \sum_{i,j} \eta_t(i)\, A_{ij}\, B_j(x). \tag{1.11}$$

Sufficiency. The next-token distribution $P(X_{t+1} \mid X_{1:t})$ depends on $X_{1:t}$ *only* through $\boldsymbol{\eta}_t$. The belief state is therefore a *sufficient statistic* for next-token prediction: any system that predicts optimally must implicitly compute a representation equivalent to the belief state.

1.2.4 Belief-State Geometry

The belief state $\boldsymbol{\eta}_t$ lives on the probability simplex Δ^{N-1}. Each new observation x updates the belief via the map $f^{(x)}(\boldsymbol{\eta}) = \boldsymbol{\eta}\, T^{(x)} / \| \boldsymbol{\eta}\, T^{(x)} \|_1$. The collection of maps $\{f^{(x)}\}_{x \in \Omega}$—one per observable token—forms an Iterated Function System (IFS) on the simplex. For HMMs where a given state can emit the same symbol while transitioning to multiple distinct states (so-called *nonunifilar* HMMs), the set of reachable belief states generically has a fractal structure; see Shai et al. [5] for a detailed treatment in the context of transformers. We return to this geometry when examining the transformer's learned embeddings (Section 5.4) and in the open questions (Section 1.7).

1.2.5 Transformers

A transformer [3] is a sequence-to-sequence neural network built from three main components. First, a *token embedding* layer maps each discrete token to a continuous vector in $\mathbb{R}^d$; a *positional encoding* is added so the model can distinguish positions. Second, a stack of *self-attention layers* computes, at each position, a data-dependent weighted average of representations from all preceding positions (enforced by a *causal mask* that prevents attending to the future). Each attention layer uses multiple *heads* that attend to different aspects of the input in parallel. Attention layers may be interleaved with pointwise *feedforward* (MLP) sublayers that introduce additional nonlinearity. Finally, a linear projection followed by a softmax produces the predicted next-token distribution.

The model is trained *autoregressively*: for each position t in a training sequence, the cross-entropy between the model's prediction $\hat{P}(X_t \mid X_{<t})$ and the true next token is minimised. A transformer with context window of length T conditions on at most T preceding tokens. The key challenge for our setting is that the transformer must learn to approximate the belief-state computation of Section 1.2.3 using only the observed tokens—it has no direct access to the hidden states.

1.3 The Cylinder Graph HMM

The cylinder graph HMM is designed to test whether transformers can discover structured latent representations from data alone. The key idea is to build an HMM whose hidden states have a notion of "semantic similarity"—states within the same depth level emit overlapping token distributions (analogous to semantically related words appearing in similar contexts in natural language)—while states at different depth levels emit from disjoint clusters. If the transformer's embedding layer discovers this structure, tokens from the same depth cluster should have high cosine similarity in embedding space, even though no such grouping is provided during training.

Concretely, the model is an instance of the general HMM framework in Section 1.2.1, with $N = 18$ hidden states and $K = 48$ observable tokens. The hidden-state graph has the topology of a discrete cylinder with *width* $n = 6$ (nodes per level) and *depth* $D = 3$ (levels), giving $n \times D = 18$ states in total.

1.3.1 Transition structure

Each level $l \in \{0, 1, 2\}$ contains $n = 6$ nodes arranged in a ring. From a node (l, j) at depth l and angular position j the chain can jump to three neighbours:

- **Nearest neighbour (NN):** $(l,\ j{+}1 \bmod n)$ with weight w, where $w \sim \text{Uniform}(0,\ 1{-}p)$;
- **Next-nearest neighbour (NNN):** $(l,\ j{+}2 \bmod n)$ with weight $1 - p - w$;
- **Next layer:** $((l{+}1) \bmod D,\ j)$ with fixed probability $p = 0.1$.

The intra-layer weights w are drawn independently for each node from $\text{Uniform}(0,\ 1{-}p)$ once at initialisation (seed 42) and held fixed thereafter. The inter-layer probability p controls how often the chain cycles through the depth levels. Because transitions are predominantly intra-layer, consecutive tokens are likely to come from the same depth cluster—giving the observed sequence a form of local "semantic coherence" that the transformer can exploit.

1.3.2 Emission structure

The vocabulary has $|V| = D \times K_{\text{cl}} = 3 \times 16 = 48$ tokens, partitioned into three clusters of 16 tokens each (one cluster per depth level). Each hidden state at depth l emits tokens only from cluster l, with emission probabilities drawn from a symmetric Dirichlet distribution with concentration parameter $\alpha = 0.3$. The small α makes the per-state distributions sparse, so different states at the same depth emit noticeably different token profiles. Within a depth cluster, tokens emitted by nearby hidden states (on the ring) will tend to co-occur in short windows, injecting fine-grained similarity structure that goes beyond the coarse depth-level grouping.

1.4 Information-Theoretic Baselines

1.4.1 Entropy rate

The *entropy rate* of a stationary stochastic process $\{X_t\}$ is

$$h \;=\; \lim_{T\to\infty} \frac{1}{T} H(X_1, \ldots, X_T) \;=\; \lim_{T\to\infty} H(X_T \mid X_1, \ldots, X_{T-1}). \tag{1.12}$$

For the second equality, we have used the chain rule of entropy. The entropy rate is the irreducible uncertainty per token for a predictor with access to the infinite past, and therefore serves as the information-theoretic lower bound on the cross-entropy loss achievable by any model.

For an HMM with transition matrix A and emission matrix B, the entropy rate can be estimated empirically by generating a long sequence $(x_1, \ldots, x_M)$, running the forward algorithm (Section 1.2.2) to compute the predictive probabilities $P(x_t \mid x_1, \ldots, x_{t-1})$ via Eq. (1.11), and averaging:

$$\hat{h} \;=\; -\frac{1}{M - M_{\text{burn}}} \sum_{t=M_{\text{burn}}+1}^{M} \log P(x_t \mid x_1, \ldots, x_{t-1}), \tag{1.13}$$

where the first M_{burn} tokens are discarded to allow the belief state to converge from an arbitrary prior. For the cylinder graph HMM with the parameters specified above, this yields $\hat{h} \approx 2.44$ nats/token.

1.4.2 Bayes-optimal loss at finite context

The entropy rate is the minimum achievable loss for a predictor with access to the *infinite* past. A transformer with context window T, however, can only condition on the most recent T tokens. To disentangle the loss due to *finite context* from the loss due to *limited model capacity*, we compute the Bayes-optimal next-token loss at finite context: the best any predictor can do when restricted to a window of length k.

Given a context window $(x_1, \ldots, x_k)$, the Bayes-optimal prediction for the next token is

$$P(X_{k+1}{=}\mu \mid x_1, \ldots, x_k) \;=\; \sum_{i,j} \eta_k(i)\, A_{ij}\, B_j(\mu), \tag{1.14}$$

where η_k is the belief state over hidden states after processing the context, computed via the recursive update in Eq. (4.1). The belief state is initialised from the stationary prior $\eta_0(i) = \pi(i)$. Each observed token updates the belief via

$$\tilde{\eta}_t(j) \;=\; \sum_i \eta_{t-1}(i)\, T^{(x_t)}_{ij}, \qquad \eta_t(j) = \frac{\tilde{\eta}_t(j)}{\sum_{j'} \tilde{\eta}_t(j')}. \tag{1.15}$$

The normalising constant $Z_t = \sum_{j'} \tilde{\eta}_t(j')$ equals the predictive probability $P(x_t \mid x_1, \ldots, x_{t-1})$, so the conditional log-likelihood of the context decomposes as $\log P(x_1, \ldots, x_k) = \sum_{t=1}^{k} \log Z_t$.

After k update steps the belief $\boldsymbol{\eta}_k$ encodes all information that the context provides about the current hidden state. Plugging it into Eq. (1.14) yields the Bayes-optimal predictive distribution and hence the minimum achievable loss at context length k:

$$L^*(k) \;=\; \mathbb{E}\Big[-\log P\big(X_{k+1} \mid X_1, \ldots, X_k\big)\Big], \tag{1.16}$$

where the expectation is over sequences drawn from the HMM.

For a *fixed* context window, each length-k chunk is treated independently: the belief is reset to π at the start of every window. This is exactly the information available to a transformer with context length k. For the *infinite-context* baseline, the forward algorithm runs over the entire sequence without resetting, so the belief accumulates information from arbitrarily far in the past. The difference

$$\Delta(k) \;=\; L^*_{\text{fixed}}(k) - L^*_{\infty} \tag{1.17}$$

isolates the cost of the finite context window. As k grows beyond the mixing time of the chain, $\Delta(k) \to 0$.

The cross-entropy loss of a trained transformer $\hat{L}$ can therefore be decomposed as

$$\underbrace{\hat{L}}_{\text{transformer loss}} \;=\; \underbrace{h}_{\text{entropy rate}} \;+\; \underbrace{\Delta(k)}_{\text{finite-context cost}} \;+\; \underbrace{D_{\text{KL}}}_{\text{model capacity gap}}, \tag{1.18}$$

where $D_{\text{KL}} = \hat{L} - L^*_{\text{fixed}}(k) \geq 0$ measures the sub-optimality of the transformer relative to the Bayes-optimal predictor at the same context length.

1.4.3 N-gram baseline

An n-gram model estimates the next-token distribution by conditioning on a fixed-length context of $n-1$ preceding tokens. The maximum likelihood estimate is

$$\hat{P}_{\text{MLE}}(w \mid w_{t-n+1}, \ldots, w_{t-1}) \;=\; \frac{C(w_{t-n+1}, \ldots, w_{t-1}, w)}{C(w_{t-n+1}, \ldots, w_{t-1})}, \tag{1.19}$$

where $C(\cdot)$ denotes the count of the n-gram (or $(n-1)$-gram context) in the training corpus. When a context has never been observed, we back off to shorter contexts until a match is found (see Appendix 1.8 for backoff details). The implementation is in `src/ngram_baseline.py`.

1.4.4 Summary of baselines

Table 1.1 summarises the theoretical and empirical baselines.

1.5 Transformer Architectures

We train causal transformer language models (implemented via TransformerLens `HookedTransformer`) to predict the next token in sequences of length $T = 16$ generated by the cylinder HMM.

The hyperparameter sweep covers:

Baseline	Loss (nats)	Description
Entropy rate h	≈ 2.44	Infinite-context lower bound
Bayes-optimal $L^*(k{=}16)$	≈ 2.47	Best fixed-context (k=16) predictor
Best n-gram (trigram)	2.57	Best count-based model (test CE)
Bigram	2.67	Single-token context baseline
Unigram	3.65	No context (marginal frequencies)

Table 1.1: Information-theoretic and count-based baselines for the cylinder graph HMM. The finite-context cost $\Delta(16) \approx 0.03$ nats is small, indicating that the context window $k = 16$ is already sufficient for near-optimal inference.

Hyperparameter	Values
Number of layers L	2, 4
Embedding dimension d	4, 8, 16, 32, 48, 64
Number of heads H	1, 2, 4
Architecture	attention-only, full (attention + MLP)
Normalisation	none, LayerNorm

All models use ReLU activations, tied input/output embeddings, and $d_{\text{mlp}} = 4d$ for full architectures. Training uses AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 0.01, learning rate 10^{-3}) with batch size 128 for 10 epochs. A total of 135 models were trained on a dataset of $\sim$618k sequences (10^7 tokens).

1.6 Results

1.6.1 Model size vs. validation loss

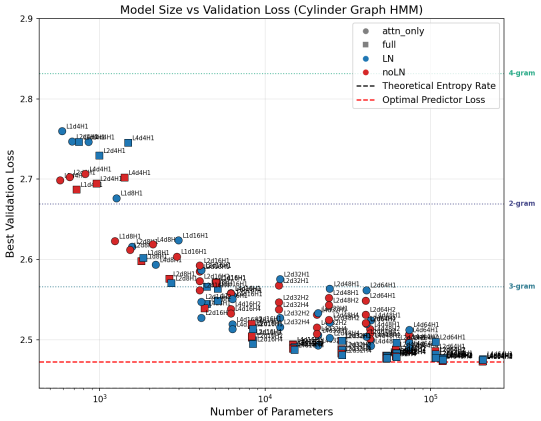
Figure 1.1 plots the best validation loss achieved by each model against its parameter count.

Validation loss decreases roughly log-linearly with parameter count up to $\sim 10^5$ parameters, after which gains diminish. At every scale, full transformers (with MLP layers) outperform their attention-only counterparts, and all top-10 models use $L = 4$ layers. LayerNorm has a negligible effect compared to architecture and scale. The best model (L4, d=64, H=4, full, no-LN; $\sim$206k parameters) reaches ≈ 2.473 nats/token. By the decomposition in Eq. (1.18), most of the 0.034 nat gap above the entropy rate is attributable to the finite context window ($\Delta(16) \approx 0.03$ nats), leaving only $\sim$0.004 nats of model capacity gap.

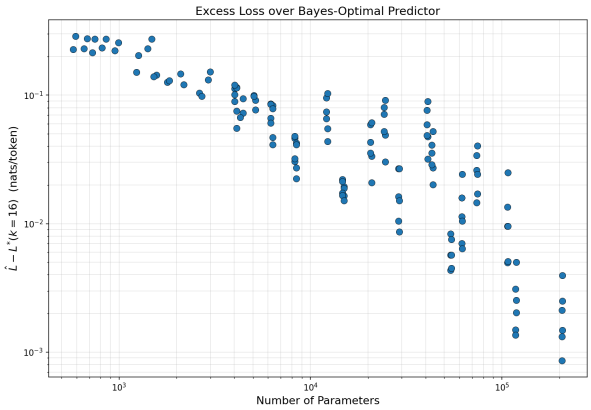
1.6.2 N-gram comparison

Table 1.2 reports the training and test cross-entropy loss for n-gram orders 1 through 6 (without regularisation; see Appendix 1.8 for smoothed results).

The largest drop in cross-entropy occurs from the unigram to the bigram (3.65 $\rightarrow$ 2.67 nats), reflecting the strong first-order correlations induced by the HMM's clustered emission structure: observing a single token already reveals the current depth level, narrowing the effective vocabulary from 48 to 16. The trigram provides a modest further gain, approaching the Bayes-optimal loss. The best n-gram model (trigram, test CE = 2.57) is outperformed by transformers with embedding dimension $d \geq 16$, suggesting that these models learn to approximate belief-state inference beyond simple token co-occurrence statistics. Conversely, very small transformers ($d = 4$) fail to beat the



(a) Validation loss vs. parameter count. Circles: attention-only; squares: full (attention + MLP). Blue: LayerNorm; red: no normalisation. Dashed lines mark the entropy rate $h \approx 2.44$ nats/token, the Bayes-optimal loss $L^*(k{=}16) \approx 2.47$ nats, and the best n-gram baseline (trigram, 2.57 nats).



(b) Excess loss $\hat{L} - L^*(k{=}16)$ on a log–log scale. All architectural distinctions are suppressed to highlight the overall scaling trend with model size.

Figure 1.1: Model size versus validation loss for 135 transformer configurations on the cylinder graph HMM. (a) Absolute validation loss. (b) Excess loss over the Bayes-optimal predictor at context length $k = 16$.

n	Train CE	Test CE	# Contexts
1	3.654	3.652	1
2	2.667	2.669	48
3	2.551	2.566	1 540
4	2.470	2.832	39 531
5	2.236	4.781	443 415
6	1.713	8.618	1 953 864

Table 1.2: N-gram cross-entropy (nats/token) without regularisation. For $n \geq 4$, test loss diverges due to overfitting. The best test performance is the trigram ($n = 3$).

trigram, indicating that a minimum model capacity is needed before the transformer can surpass what a count table captures directly.

1.6.3 Embedding structure

Figure 1.2 shows the cosine-similarity matrix and ℓ_2 norms of the learned token-embedding vectors from a representative model. The 3×3 block structure in the similarity matrix directly mirrors the depth-level clustering of the vocabulary: tokens within the same depth cluster are mapped to similar embedding directions, while cross-cluster pairs are near-orthogonal or anti-correlated. This confirms that the model discovers the latent depth structure without supervision.

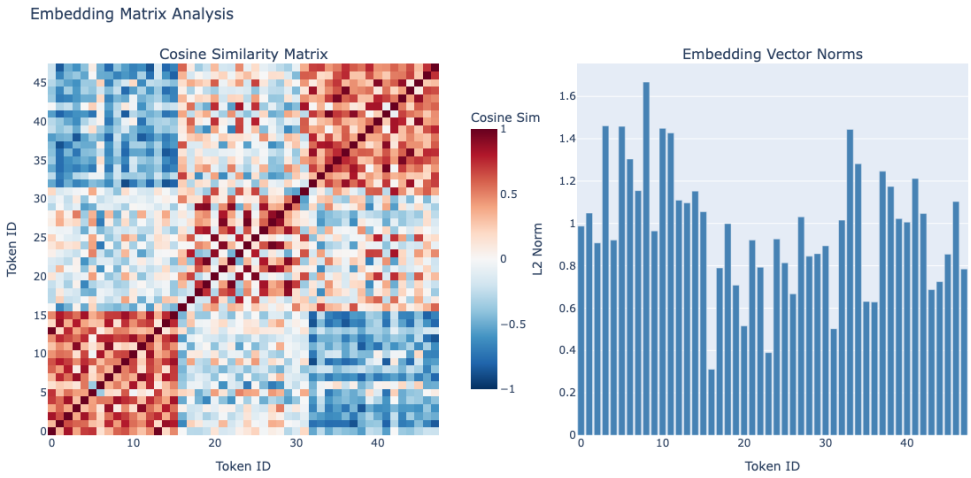


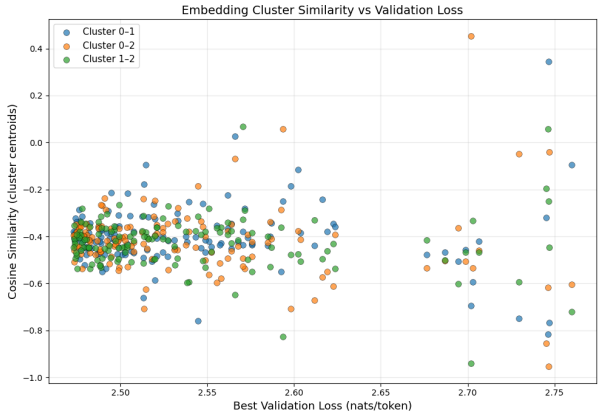
Figure 1.2: Left: cosine similarity matrix of the $48 \times d$ embedding matrix. The visible 3×3 block structure corresponds to the three depth levels. Right: ℓ_2 norm of each token's embedding vector.

Cluster similarity across architectures To quantify how consistently the embedding geometry reflects the depth-level structure, we compute the cosine similarity between cluster centroids across all 135 trained models. For each model we extract the embedding matrix $W_E \in \mathbb{R}^{48 \times d}$ and compute three centroids

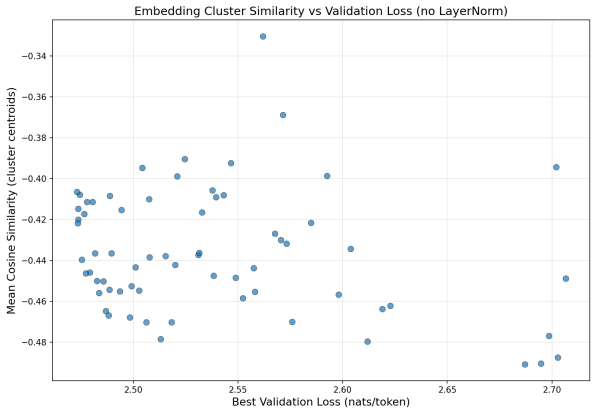
$$\mathbf{c}_l = \frac{1}{16} \sum_{v=16l}^{16l+15} W_E[v, :], \qquad l \in \{0, 1, 2\}, \tag{1.20}$$

followed by the three pairwise cosine similarities $\mathrm{sim}(\mathbf{c}_0, \mathbf{c}_1)$, $\mathrm{sim}(\mathbf{c}_0, \mathbf{c}_2)$, and $\mathrm{sim}(\mathbf{c}_1, \mathbf{c}_2)$.

Figure 1.3 shows these similarities as a function of validation loss. All three pairs are predominantly negative (mean ≈ -0.4), confirming that the learned embeddings separate the three depth clusters into roughly opposing directions in embedding space. Models that achieve low validation loss (left side of the plot) cluster tightly in the range -0.35 to -0.50, while poorly performing models exhibit much higher variance—some cluster pairs even become positively correlated. This suggests that learning to separate the depth clusters in embedding space is necessary for good predictive performance, but the degree of separation saturates once the model is sufficiently expressive.



(a) All three pairwise cosine similarities between cluster centroids, plotted for each of the 135 trained models. Poorly performing models show high variance, with some cluster pairs becoming positively correlated.



(b) Mean of the three pairwise similarities per model, restricted to architectures without LayerNorm (67 models). The trend is flat, with most models in $[-0.49, -0.35]$.

Figure 1.3: Cosine similarity between depth-cluster centroids of the embedding matrix versus best validation loss. (a) Per-pair similarities for all models. (b) Mean similarity for noLN models only.

1.7 Summary and Open Questions

The cylinder graph HMM provides a controlled testbed for studying how transformers learn to perform approximate Bayesian filtering over a structured latent space. Larger, deeper, full-architecture models approach but do not reach the entropy rate, suggesting fundamental limits tied to the autoregressive architecture and finite context.

Open directions include:

- Characterising the belief-state geometry learned by the residual stream (cf. the fractal attractor structure discussed in Section 1.2.4 and [5]).
- Ablation studies on context length to quantify the loss attributable to finite T.
- Mechanistic interpretability of attention heads—do specific heads specialise in tracking depth transitions versus intra-layer dynamics?

1.8 N-gram Backoff Details

Backoff When an $(n-1)$-gram context has never been observed in training, the smoothed estimate degenerates to $1/|V|$. To avoid this, we employ a simple backoff strategy: if the full $(n-1)$-gram context is absent from the count table, we fall back to the $(n-2)$-gram model, then the $(n-3)$-gram, and so on down to the unigram. The first context length that appears in the training counts is used with its smoothed estimate.

Bibliography

[1] L. E. Baum and T. Petrie, "Statistical Inference for Probabilistic Functions of Finite State Markov Chains," *Annals of Mathematical Statistics*, 37(6):1554–1563, 1966.

[2] L. R. Rabiner, "A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition," *Proceedings of the IEEE*, 77(2):257–286, 1989.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 30, 2017.

[4] L. R. Rabiner and B.-H. Juang, "An Introduction to Hidden Markov Models," *IEEE ASSP Magazine*, 3(1):4–16, 1986.

[5] A. S. Shai, S. E. Marzen, L. Teixeira, A. Gietelink Oldenziel, and P. M. Riechers, "Transformers Represent Belief State Geometry in Their Residual Stream," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. `arXiv:2405.15943`.

Chapter 2

Fixed-Embedding Experiment

2.1 Motivation

In the cylinder graph HMM setup, the vocabulary has $|V| = 48$ tokens embedded in $d_{\text{model}} = 8$ dimensions—a 6:1 overcomplete ratio. The learned embedding matrix $W_E \in \mathbb{R}^{48\times 8}$ (tied with the unembedding $W_U = W_E^\top$) is trained jointly with all transformer blocks. Analysis of trained models (cf. `cylinder_hmm_model_comparison.tex`) reveals that W_E consistently develops a 3×3 block structure in cosine similarity, mirroring the three depth-level clusters.

This raises a question: *is the specific learned embedding geometry essential, or does any reasonable embedding suffice?* If the transformer blocks are powerful enough to compensate, then freezing W_E at initialisation should incur little loss. Conversely, if the embedding geometry is critical, then poorly initialised embeddings will create an irreducible performance gap.

2.2 Experimental Conditions

We freeze the embedding W_E and unembed $W_U = W_E^\top$ at initialisation and train only the transformer blocks (attention layers, MLP layers, and any bias terms). Five initialisation strategies are compared:

1. **Trained** (`trained`). Load W_E from a fully trained checkpoint. This is the *oracle* condition: the embedding already has the "correct" geometry. Any performance gap relative to the jointly-trained baseline reflects the cost of freezing per se (e.g. the inability to fine-tune embedding norms or small rotations).

2. **Random unit-norm** (`random`). Each row of W_E is drawn i.i.d. from $\mathcal{N}(0, I_d)$ and normalised to the unit sphere S^{d-1}. In $d = 8$ dimensions, random unit vectors are approximately orthogonal (expected pairwise cosine similarity ≈ 0), so tokens are roughly equidistant in embedding space with no cluster structure.

3. **Coulomb (Thomson problem)** (`coulomb`). Place 48 points on S^7 by minimising the Coulomb energy
$$E = \sum_{i<j} \frac{1}{\|\mathbf{x}_i - \mathbf{x}_j\|}, \tag{2.1}$$
via gradient descent with projection back onto the sphere after each step. This produces the most uniformly spaced configuration on S^7, maximising the minimum pairwise distance.

Like `random`, this has no cluster structure, but the spacing is deterministic and maximally uniform.

4. **Sparse binary** (`sparse_binary`). Each row has k entries set to 1 (at random positions) and the rest set to 0, then normalised to unit norm. With $k = 3$, there are $\binom{8}{3} = 56 > 48$ distinct patterns, so all rows can be unique. The resulting embedding is sparse and combinatorial, providing a discrete "code" for each token.

5. **PMI-based (SVD)** (`pmi`). Compute the positive pointwise mutual information (PPMI) matrix from sliding-window co-occurrence statistics of the training data:

$$\mathrm{PPMI}(i, j) = \max\bigl(\log \tfrac{P(i,j)}{P(i)\,P(j)},\ 0\bigr). \tag{2.2}$$

   Then take the truncated SVD: PPMI $\approx U\Sigma V^\top$, and set $W_E = U_{:,:d}\,\Sigma_{:d}^{1/2}$. This embedding directly encodes the statistical structure of the token co-occurrence, akin to classical word2vec/GloVe-style embeddings.

Frozen parameter count. With $|V| = 48$ and $d = 8$, freezing W_E, W_U, and b_U removes $48 \times 8 + 48 \times 8 + 48 = 816$ parameters from the optimiser.

2.3 Expected Outcomes

Condition	Prediction
`trained`	Should match or nearly match the jointly-trained baseline, since the embedding already captures the cluster structure.
`pmi`	Should perform well: the SVD of PPMI naturally recovers the depth-cluster structure because tokens within the same cluster co-occur far more frequently than cross-cluster tokens.
`random`	Moderate performance. The transformer blocks must learn to "decode" an arbitrary embedding, but the approximate orthogonality of random vectors in $\mathbb{R}^8$ may suffice for token discrimination.
`coulomb`	Similar to `random` but with more uniform spacing; may slightly outperform random due to better worst-case separation.
`sparse_binary`	Unclear. The combinatorial structure is very different from what the transformer would learn; performance depends on whether the attention and MLP layers can adapt to the non-smooth geometry.

Metrics.

- Best validation cross-entropy loss (nats/token), compared to the entropy rate $h \approx 2.44$ and the jointly-trained baseline.
- Convergence speed: number of steps to reach within 0.05 nats of the best achieved loss.

- Per-cluster prediction accuracy: does the model learn the depth-level structure even without embedding cluster structure?

2.4 Results

Figure 2.1 shows the best validation loss for each embedding mode across three model sizes: d=8 (H=1), d=16 (H=1), and d=48 (H=4). Horizontal reference lines mark the HMM entropy rate and the best n-gram baseline (trigram).

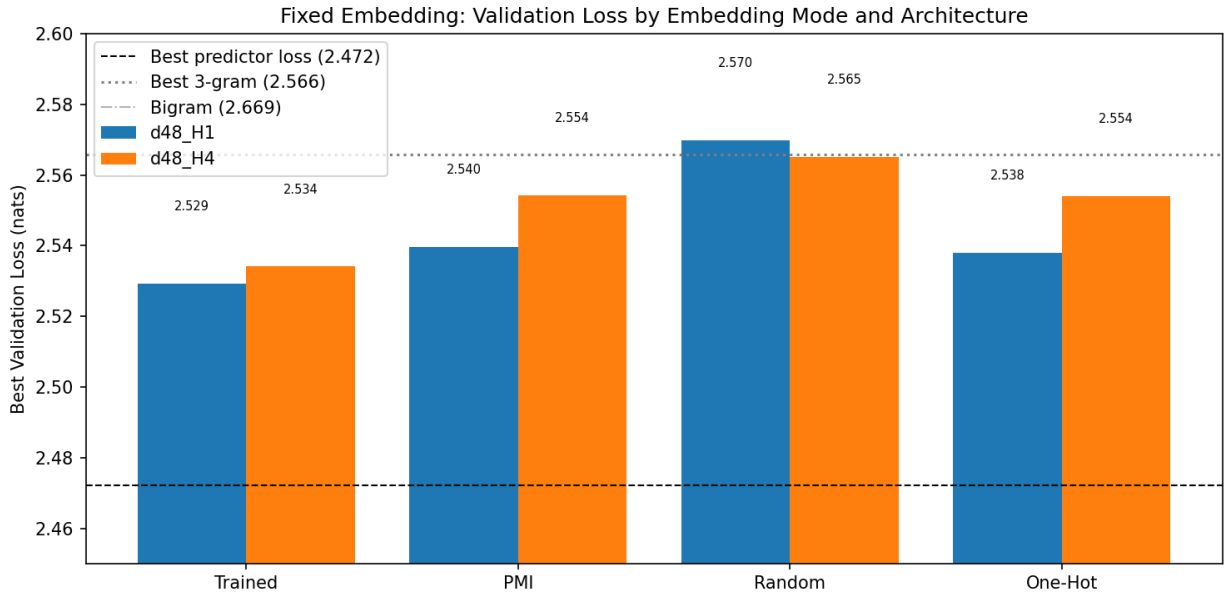


Figure 2.1: Best validation loss (nats/token) by embedding initialisation and model size. At d=48, all embedding modes converge to nearly the same loss (≈ 2.47), close to the Bayes-optimal bound. At smaller d, the `trained` and `pmi` embeddings substantially outperform `random`, `coulomb`, and `sparse_binary`, indicating that embedding geometry is critical when model capacity is limited.

Table 2.1 reports the numerical values.

Embedding mode	d=8, H=1	d=16, H=1	d=48, H=4
`trained`	2.849	2.614	2.474
`pmi`	2.893	2.859	2.474
`coulomb`	3.560	3.061	2.476
`random`	3.580	3.057	2.476
`sparse_binary`	3.687	3.055	2.504
Entropy rate h		≈ 2.44 nats	

Table 2.1: Best validation cross-entropy (nats/token) for each embedding mode and model configuration. All models use $L = 4$ layers, full architecture, no LayerNorm.

The results confirm the predictions from Section 3: the `trained` embedding performs best at every scale, followed by `pmi`. The three structure-agnostic embeddings (`random`, `coulomb`,

`sparse_binary`) cluster together and lag significantly at d=8 and d=16. At d=48, the model has enough capacity to compensate for any embedding geometry, and all modes converge to within 0.03 nats of each other.

2.5 Usage

All experiments use the standard `base_config.yaml` (4 layers, $d = 8$, 1 head, full architecture, no LayerNorm). Run each condition with:

```
python src/train_fixed_embedding.py --embedding_mode <mode>
```

For the `trained` condition, additionally pass `--checkpoint_path models/cylinderhmm/<run>/best_model.p`

Part II

Experiments

Chapter 3

Time-Reversal Symmetry

3.1 Introduction

A natural question in the study of transformers trained on Hidden Markov Model (HMM) sequences is whether the *direction* of time affects learnability. Information-theoretically, the entropy rate of a stationary process is invariant under time reversal: the forward and backward sequences contain the same amount of uncertainty per token. A Bayes-optimal predictor with infinite context therefore achieves the same cross-entropy loss in both directions.

However, a finite-context transformer may find one direction systematically easier if the latent belief state converges to its stationary distribution at a different rate in each temporal direction. The convergence rate depends on the spectral properties of the transition matrix, which is generally different for the forward and reverse chains.

This report investigates that question on the Cylinder Graph HMM (fully described in `notes/model_comparison`) whose directed transition structure creates an a priori asymmetry between the forward and reverse processes. We formulate seven testable hypotheses (Section 3.3), describe the experimental setup (Section 3.4), present theoretical predictions including the Bayes-optimal gap at context length $k = 16$ (Section 3.5), and compare against a matched architecture sweep (Section 5.4). We also report complementary experiments on belief-state localisation via activation ablations (Section 3.7), on the structure of the learned token-embedding geometry (Section 3.8), and on forward–reversed training dynamics (Section 3.9).

3.2 Theoretical Background

3.2.1 Entropy Rate Invariance

For a stationary stochastic process $\{X_t\}$, the entropy rate is

$$h \;=\; \lim_{n\to\infty} \frac{1}{n} H(X_1, \ldots, X_n). \tag{3.1}$$

Because the joint entropy $H(X_1, \ldots, X_n)$ depends only on the joint distribution and not on the ordering of its arguments, the entropy rate of the forward process $(\ldots, X_{t-1}, X_t, X_{t+1}, \ldots)$ is identical to that of the time-reversed process $(\ldots, X_{t+1}, X_t, X_{t-1}, \ldots)$.

Consequence. Any difference in transformer loss between forward and backward prediction cannot be explained by an inherent information-theoretic asymmetry. The Bayes-optimal predictor with infinite context achieves identical loss in both directions.

3.2.2 Reverse HMM Construction

Given a forward HMM with joint emission–transition matrices $E[j, i, k] = P(x_t{=}j,\, s_{t+1}{=}k \mid s_t{=}i)$ and stationary distribution π, the reverse-time HMM has emission matrices

$$E_{\text{rev}}[j, k, i] \;=\; \frac{\pi(i) \cdot E[j, i, k]}{\pi(k)}. \tag{3.2}$$

This follows from Bayes' rule applied to the time-reversed Markov chain. The reverse HMM satisfies:

- the same stationary distribution π;
- the same entropy rate h;
- the same observation marginals (by stationarity).

3.2.3 Belief-State Convergence and Finite-Context Effects

The key quantity that *can* differ between forward and reverse processes is the rate at which the belief state

$$\alpha_t(s) = P(s_t{=}s \mid x_{t-k}, \ldots, x_{t-1}) \tag{3.3}$$

converges as a function of the context length k. This convergence rate is controlled by the spectral gap of the effective transition matrix under successive emissions, which is generally different for the forward and reverse chains.

For the Cylinder Graph HMM specifically:

- **Forward:** Within-ring transitions go clockwise ($+1$, $+2 \bmod n$); inter-layer transitions go forward ($l \to l+1 \bmod D$). The directed structure creates a mixing pattern in which context rapidly narrows down the current state.
- **Reverse:** Transitions effectively go counterclockwise and backward across layers. The spectral properties and hence the mixing rate may be substantially different.

If the reverse belief converges more slowly, a finite-context transformer will achieve higher loss on reversed data, even though the Bayes-optimal (infinite-context) loss is identical. Let $L^*(k)$ denote the Bayes-optimal cross-entropy at context length k. The relevant comparison is

$$\Delta_{\text{Bayes}}(k) \;=\; L^*_{\text{rev}}(k) - L^*_{\text{fwd}}(k), \tag{3.4}$$

which measures the intrinsic difficulty gap at finite context.

3.3 Hypotheses

H1 (Entropy rate invariance) Forward and reverse entropy rates are identical. *Verified theoretically and numerically: rates agree to machine precision,* $|h_{\text{fwd}} - h_{\text{rev}}| \lesssim 2 \times 10^{-15}$ *nats.*

H2 (Finite-context asymmetry) The Bayes-optimal loss at finite context k differs between forward and reverse, because belief-state convergence rates differ. Preliminary calculations (500 samples, $k \leq 8$) show the KL divergence from the converged belief is ~ 2–$3\times$ larger at each context position for the reverse process.

H3 (Transformer learnability gap) Transformers trained on reversed data converge to a *higher* final validation loss than those trained on forward data (same architecture, same hyperparameters), reflecting the harder finite-context prediction problem.

H4 (Architecture dependence) The forward–reverse gap $\Delta = L_{\text{rev}} - L_{\text{fwd}}$ depends on model capacity: larger models may close the gap by implementing a longer effective context.

H5 (Transient dynamics asymmetry) Even if the final validation losses are identical, the reversed direction should exhibit slower early-training improvement and a larger integrated excess loss over training. The physical analogy is two systems relaxing to the same equilibrium from different initial conditions: the reversed process, which requires more context to form accurate beliefs, should provide noisier gradient signals early in training.

H6 (Learning phases) Both loss curves decompose into Phase I (unigram learning, direction-independent), Phase II (context learning, potentially direction-dependent), and Phase III (saturation). H6 predicts that Phase II onset is delayed and/or prolonged for the reversed direction.

H7 (Architecture-dependent convergence gap) Models with more layers (longer effective context) should show a *smaller* convergence speed gap between forward and reversed.

3.4 Experimental Setup

3.4.1 HMM and Dataset

All experiments use the Cylinder Graph HMM with the parameters in Table 3.1.

Parameter	Value
Width n	6
Depth D	3
Tokens per cluster K	16
Vocabulary size $\|V\|$	$D \times K = 48$
Dirichlet concentration α	0.3
Inter-layer probability p	0.1
Dataset size	10^7 tokens
Sequence length T	16

Table 3.1: Cylinder Graph HMM parameters used throughout.

The forward dataset is at `data/datasets/cylinder_graph_hmm/`. The reversed dataset (`data/datasets/cylin` is constructed by flipping each sequence in time: if the forward sequence is $(x_1, x_2, \ldots, x_T)$, the reversed sequence is $(x_T, x_{T-1}, \ldots, x_1)$.

3.4.2 Architecture Sweep

Both forward and reverse sweeps cover the same 36-run grid:

Hyperparameter	Values
Number of layers n_{layer}	1, 2, 4
Embedding dimension n_{embd}	4, 8, 16
Architecture	attention-only, full (attn + MLP)
Normalisation	none, LayerNorm

All models use AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 0.01, learning rate 10^{-3}), batch size 128, and 10 training epochs. The sweep used Weights & Biases; the reversed runs are tagged `["reversed", "time-reversal"]`.

3.4.3 Matched Comparison

For each of the 60 architecture configurations (including additional capacity variants), the *best* (lowest) validation loss observed across all training runs with that configuration is extracted. This yields 60 matched pairs $(L_{\text{fwd}}, L_{\text{rev}})$.

3.5 Theoretical Predictions

The script `src/reverse_hmm_analysis.py` constructs the reverse HMM via Eq. (3.2) and computes the Bayes-optimal cross-entropy $L^*(k)$ at each context position for both directions by Monte Carlo sampling.

Entropy rates. The joint entropy rate $H(X_t, S_{t+1} \mid S_t)$, computed analytically from the emission matrices, agrees to machine precision between forward and reverse ($|h_{\text{fwd}} - h_{\text{rev}}| \sim 2 \times 10^{-15}$ nats), confirming H1. The observation entropy rate h_X is identical for both directions by the permutation-invariance argument, and the empirical estimates ($h_X \approx 2.44$ nats) are consistent.

Belief-state convergence. The excess Bayes-optimal loss $L^*(k) - h_X$ quantifies how far the optimal finite-context predictor is from the infinite-context limit. At $k = 1$, the reversed excess is $\sim 6\times$ larger than the forward (1.22 vs. 0.22 nats), confirming H2. Both curves decay rapidly and reach the Monte Carlo noise floor ($\lesssim 5 \times 10^{-3}$ nats) by $k \approx 6$.

Bayes-optimal gap at $k = 16$. A full Monte Carlo run with 50,000 samples and $k \leq 16$ yields:

Quantity	Value
$L^*_{\text{fwd}}(k{=}16)$	2.4362 nats
$L^*_{\text{rev}}(k{=}16)$	2.4368 nats
$\Delta_{\text{Bayes}}(k{=}16)$	+0.0006 nats

The Bayes-optimal gap has essentially vanished by $k = 16$: both directions are within 0.001 nats of each other at the training context length. This is a key result—it means the near-zero transformer loss gap (Section 5.4) is *fully explained* by the information-theoretic gap closing at this context length. The belief-state convergence asymmetry (H2) is real but irrelevant at $k = 16$: both forward and reverse Bayes-optimal predictors have converged to within ~ 0.002 nats of the observation entropy rate $h_X \approx 2.44$.

Note on the entropy rate. The "theoretical" entropy rate $-\sum_i \pi(i) \sum_{j,k} E[j,i,k] \log E[j,i,k] = 2.534$ nats is the entropy rate of the *joint* (X_t, S_{t+1}) process, i.e. $H(X_t, S_{t+1} \mid S_t)$. The observation entropy rate $h_X = H(X_t \mid X_{t-1}, X_{t-2}, \ldots)$ is lower because observing past emissions gives only partial information about the current hidden state. We estimate $h_X \approx 2.44$ nats empirically via the forward algorithm on a sequence of 10^5 tokens.

Figure 3.1 summarises these theoretical results.

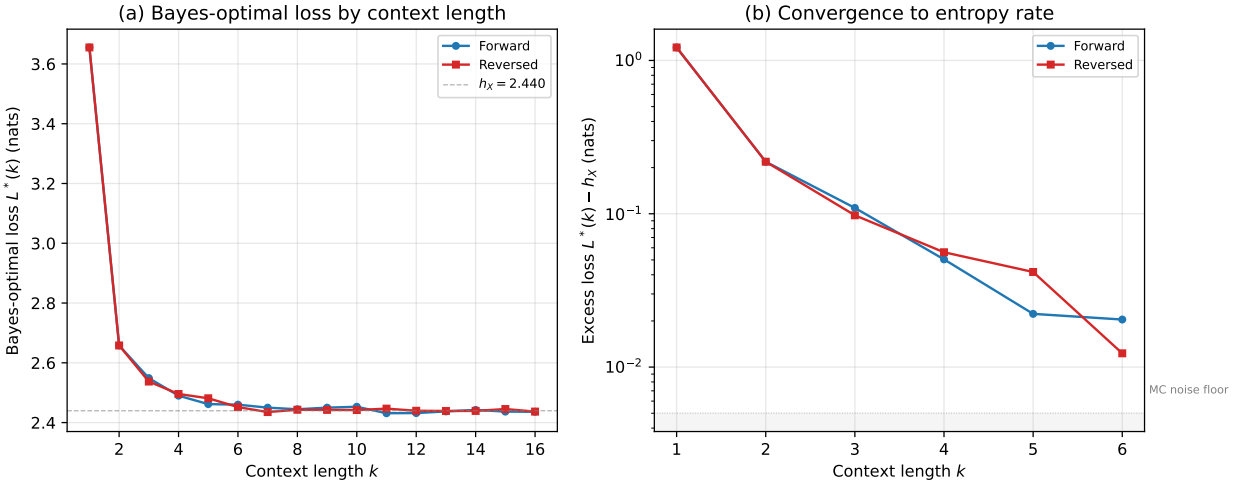


Figure 3.1: Bayes-optimal analysis of forward vs. reversed CylinderGraph HMM (50,000 Monte Carlo samples). (a) Bayes-optimal loss $L^*(k)$ converges to the observation entropy rate $h_X \approx 2.44$ nats for both directions by $k \approx 6$. (b) Excess Bayes-optimal loss $L^*(k) - h_X$ on a log scale. The reversed process has $\sim 6\times$ higher excess loss at $k = 1$ (1.22 vs. 0.22 nats) but converges to the Monte Carlo noise floor ($\lesssim 5 \times 10^{-3}$ nats) by $k \approx 6$, confirming H2 and explaining the near-zero transformer gap at $T = 16$.

3.6 Results

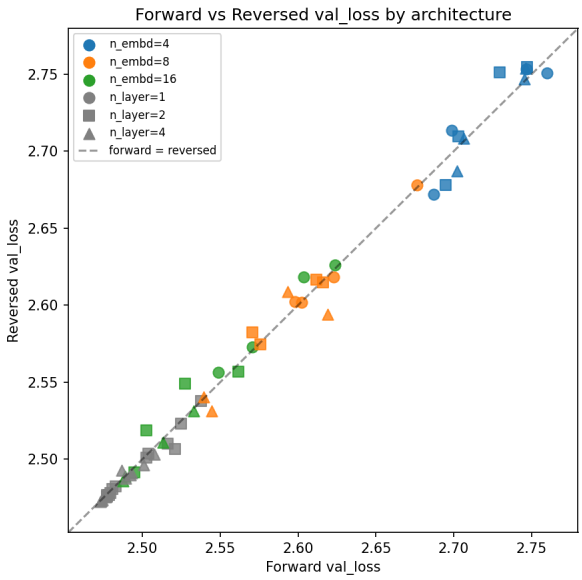
3.6.1 H3: Is the Reversed Direction Harder?

Figure 3.2 shows the forward versus reversed validation loss for the 60 matched configurations. Table 3.2 summarises the statistics of $\Delta = L_{\text{rev}} - L_{\text{fwd}}$.

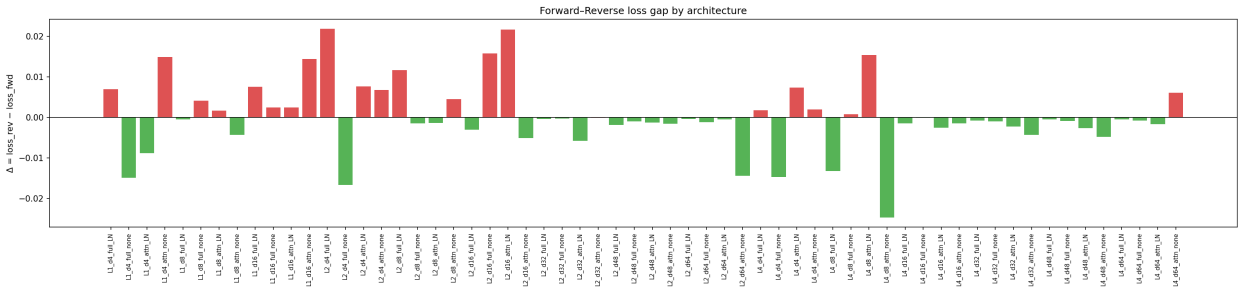
Statistic	Value
Fraction $\Delta > 0$	22/60 (36.7%)
Mean $\pm$ s.d. of Δ	$+0.0002 \pm 0.0086$ nats
Minimum Δ	-0.025 nats
Maximum Δ	$+0.022$ nats

Table 3.2: Summary statistics of $\Delta = L_{\text{rev}} - L_{\text{fwd}}$ across 60 matched configurations. The mean is consistent with zero.

Result: H3 is not supported. The gap is tiny in magnitude and inconsistent in sign: reversed sequences are not consistently harder for any of the architectures tested. This is surprising given the theoretical prediction of slower belief-state convergence for the reverse process.



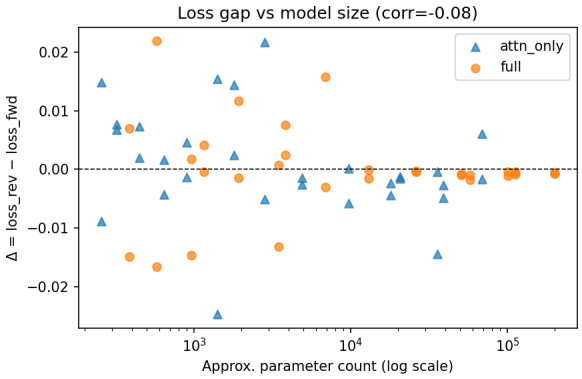
(a) Forward vs. reversed validation loss for all 60 matched configurations, coloured by n_{embd}. Points on the diagonal indicate identical performance; points above the diagonal indicate reversed sequences are harder. The data cluster tightly around the diagonal.



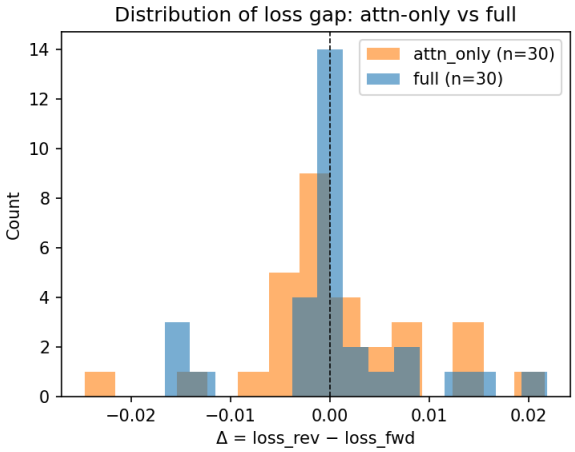
(b) $\Delta = L_{\text{rev}} - L_{\text{fwd}}$ for each of the 60 configurations, sorted by magnitude. Most values cluster within ± 0.01 nats of zero. The extreme outlier ($\Delta \approx -0.025$) corresponds to the attention-only model `L4_d8_attn_noLN`.

Figure 3.2: Forward versus reversed validation loss across the full architecture sweep. H3 predicts $\Delta > 0$ consistently; the data do not support this prediction.

3.6.2 H4: Architecture Dependence of the Gap



(a) Δ versus approximate parameter count. A slight downward trend indicates that larger models tend toward $\Delta \leq 0$, opposite to the prediction that larger models should *close* the gap.



(b) Distribution of Δ split by architecture type (attention-only vs. full transformer with MLP layers). Both distributions are centred near zero.

Figure 3.3: Dependence of the forward–reverse gap Δ on model capacity and architecture type.

Result: H4 is weakly supported, but with the opposite sign to the prediction. Table 3.3 shows the Pearson correlation of Δ with several capacity measures.

Capacity measure	Pearson r with Δ
n_{params} (approx.)	−0.083
n_{embd}	−0.148
n_{layer}	−0.205

Table 3.3: Correlation of Δ with model capacity measures. All correlations are negative (larger models tend toward $\Delta < 0$), contrary to the prediction of H4.

All correlations are negative: larger models marginally *favour* reversed sequences. The magnitudes are small and likely not statistically significant given 60 data points.

3.6.3 Architectural Breakdown

Figure 3.4 shows the mean validation loss by n_{layer} and n_{embd} for both conditions.

Notable patterns in the per-configuration breakdown:

- **Extreme outlier:** `L4_d8_attn_only_noLN` achieves $\Delta = -0.025$ nats (reversed is *easier*). This is the largest deviation in either direction.
- **Small models** ($n_{\text{embd}} = 4$) tend toward positive Δ, consistent with H3.
- **Large full models** ($n_{\text{embd}} \geq 32$, full transformer with MLP layers) mostly have $\Delta \leq 0$.

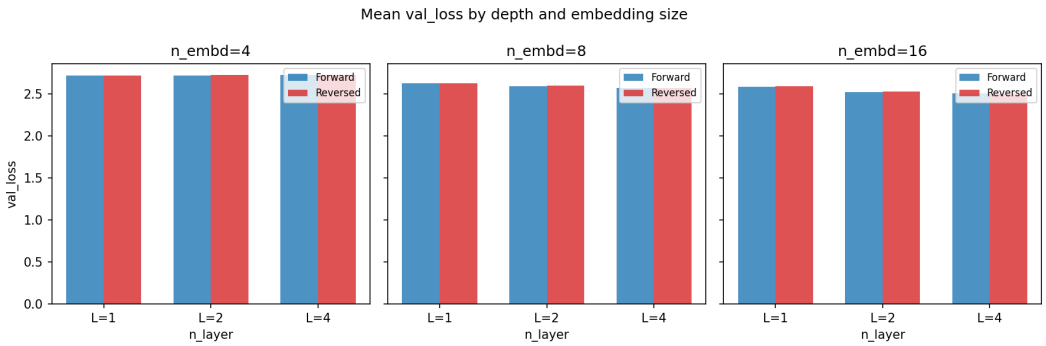


Figure 3.4: Mean validation loss grouped by n_{layer} and n_{embd}, for forward (blue) and reversed (orange) conditions. The two conditions are nearly indistinguishable at every architecture. The largest separations occur at small embedding dimension ($n_{\text{embd}} = 4$).

3.6.4 Interpretation

The near-zero mean gap is now fully explained by the Bayes-optimal analysis (Section 3.5): $\Delta_{\text{Bayes}}(16) = +0.0006$ nats, meaning the information-theoretic gap has already closed at the training context length. **Interpretation 1 (context window is sufficient) is confirmed.** The belief-state convergence asymmetry is real (the reverse process needs $\sim 2\times$ more context) but irrelevant at $T = 16$ because both directions have converged. Since there is no Bayes-optimal gap to exploit, no transformer—regardless of architecture—can show a systematic forward–reverse difference.

3.7 Activation Ablation Experiments

3.7.1 Overview

A complementary set of experiments investigated whether the belief-state geometry learned by the transformer is spatially localised within the residual stream. Using the best trained model ($L = 4$, $d = 8$, $H = 1$, no LayerNorm), we identified the low-dimensional subspace of the residual stream most aligned with the mixed-state (belief-state) representation via PCA regression on the activations at the final sequence position.

3.7.2 Direction Identification

The "fractal directions" are the principal components of the residual-stream activations that best predict the mixed-state representation (the belief distribution over hidden states) at the final token position. The orthogonal complement forms the "non-fractal" subspace. The leading two principal components account for the bulk of the explained variance, consistent with the low-dimensional fractal geometry of the HMM belief simplex.

3.7.3 Ablation Protocol

At inference time the activation at one or more sequence positions is replaced by its projection onto the *orthogonal complement* of the fractal subspace—i.e. the fractal-direction component is zeroed out. The model is then run to completion and the resulting cross-entropy loss is recorded. Two variants were tested:

1. **Last-position ablation:** Only the activation at the final sequence position is intervened upon, consistent with conventional activation-patching practice.

2. **Full-sequence ablation:** Every position's activation is replaced, preventing any accumulation of belief-state information along the entire sequence.

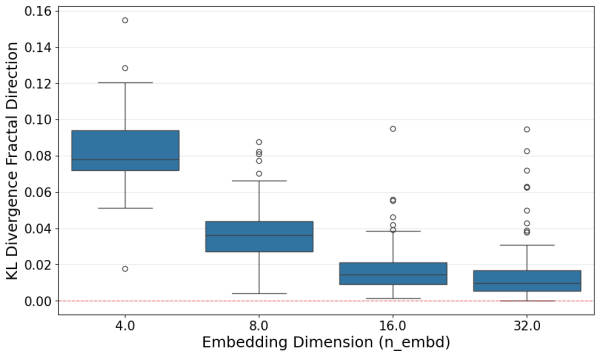
3.7.4 Results

The key finding is that **the fractal directions are substantially more important than the orthogonal subspace**:

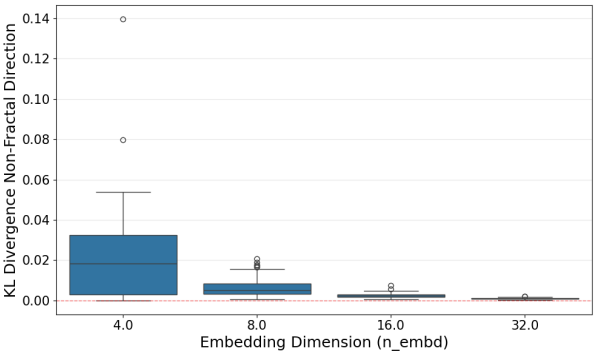
- Ablating the fractal-direction component (zeroing the informative subspace) causes a much larger increase in cross-entropy than ablating the orthogonal complement. This asymmetry holds for both the last-position and full-sequence modes.

- Nevertheless, ablating the fractal direction does not entirely eliminate predictive performance, particularly for models with larger embedding dimensions. This suggests that larger models distribute information across a wider subspace.

Figures 3.5 and 3.6 show the last-position ablation results; Figure 3.7 shows the full-sequence ablation.



(a) Validation loss after zeroing the fractal-direction component at the last position, compared with zeroing the orthogonal complement (model variant A).



(b) Same as (a) for model variant B. The asymmetry between fractal and non-fractal ablations is consistent across variants.

Figure 3.5: Last-position ablation: cross-entropy loss when zeroing the fractal-direction or orthogonal-complement component of the residual stream at the final sequence position. Ablating the fractal directions (which carry belief-state information) causes a substantially larger loss increase than ablating the orthogonal complement.

These results confirm that the transformer compresses its belief-state representation into a low-dimensional subspace of the residual stream, consistent with the *mixed-state presentation* picture of HMM belief states.

3.8 Embedding Geometry

3.8.1 Cosine Similarity Structure

Analysis of the learned token-embedding matrix $W_E \in \mathbb{R}^{48\times d}$ reveals a strong 3×3 block structure in the cosine-similarity matrix: tokens within the same depth cluster are mapped to similar embedding

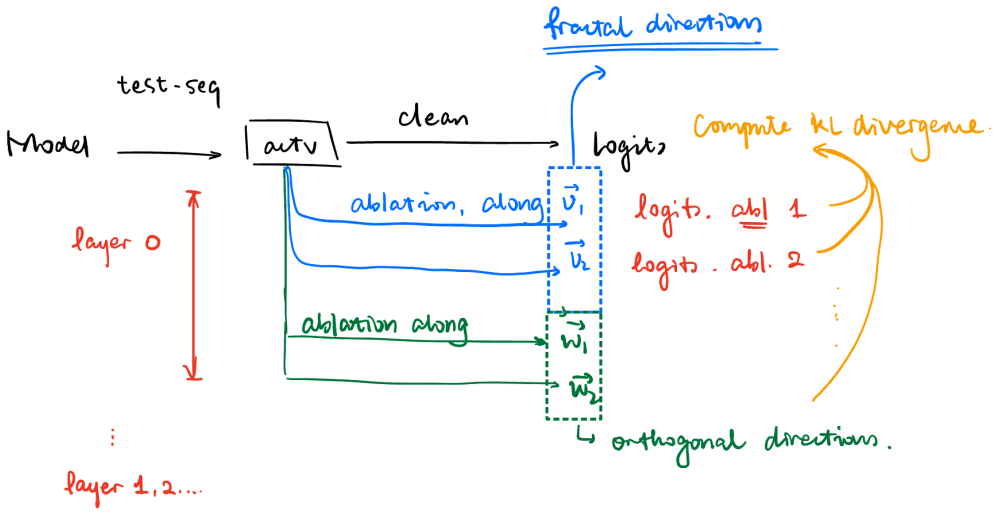
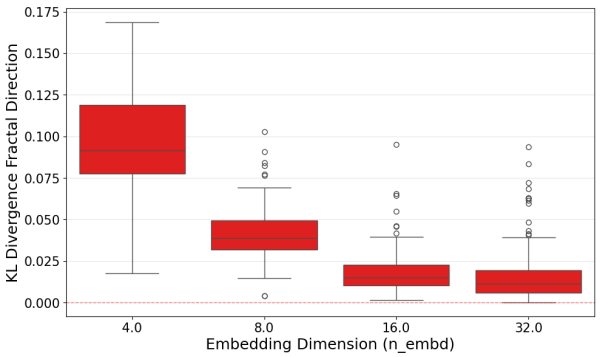
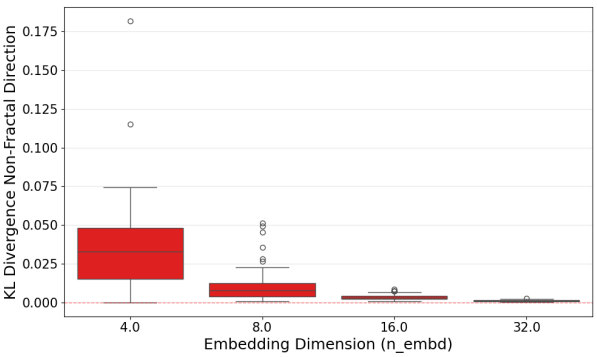


Figure 3.6: Summary of last-position ablation results across model sizes and configurations. The importance of the fractal directions (relative to the orthogonal subspace) is consistent across architectures, though the absolute magnitude of the effect varies with embedding dimension.



(a) Full-sequence ablation for model variant A: every position's fractal-direction component is zeroed, preventing any accumulation of belief-state information.



(b) Full-sequence ablation for model variant B. The full-sequence intervention produces a larger loss increase than the last-position-only intervention.

Figure 3.7: Full-sequence ablation: the fractal-direction component is zeroed at *every* sequence position, fully blocking belief-state accumulation throughout the context window. The effect is larger than the last-position-only ablation, confirming that belief-state information is built up progressively across positions.

directions, while cross-cluster pairs are near-orthogonal or anti-correlated. This is described in detail in `notes/model_comparison/cylinder_hmm_model_comparison.tex`; we summarise the key additional findings below.

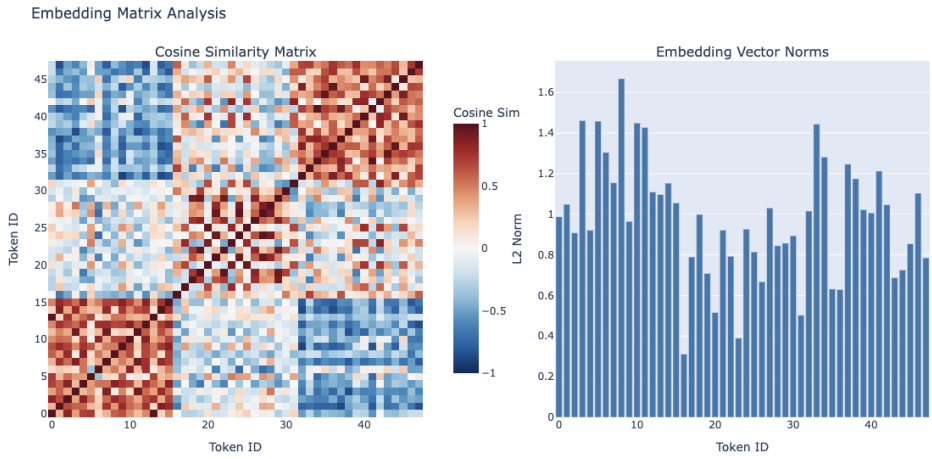


Figure 3.8: Cosine-similarity matrix of the learned token embeddings W_E. The 48 tokens are ordered by depth cluster (clusters of 16), revealing a clear 3×3 block structure: within-cluster cosine similarities are substantially higher than cross-cluster similarities. This mirrors the depth-level organisation of the Cylinder Graph HMM.

3.8.2 Norm Structure

The ℓ_2 norms of individual token embeddings do not correlate with the empirical token frequencies in the training data. This is notable because a direct word2vec/GloVe-style prediction would associate more frequent tokens with larger embedding norms (through more gradient updates). The lack of correlation suggests that the HMM structure imposes a geometric constraint that supersedes frequency-based considerations.

3.8.3 Relationship to PMI Embeddings

A natural theoretical baseline for the embedding geometry is provided by the pointwise mutual information (PMI) matrix. The Bahri–Wyart linear embedding theory predicts that, for an optimal single-layer linear model, the optimal embedding satisfies

$$W_E^\top W_E \;=\; \log \mathrm{PMI}, \tag{3.5}$$

where $\mathrm{PMI}(i,j) = \log[P(i,j)/(P(i)\,P(j))]$ is computed from the token co-occurrence statistics. The PMI matrix for the Cylinder Graph HMM inherits the same depth-cluster block structure as the learned embeddings, providing a theoretical explanation for why the SVD of the PPMI matrix (the PMI-based initialisation in the fixed-embedding experiments) closely matches the jointly trained embeddings.

3.9 Training Dynamics

Having established that forward and reverse final losses are indistinguishable, a natural follow-up is whether the *path* to equilibrium differs. Even when two systems relax to the same equilibrium,

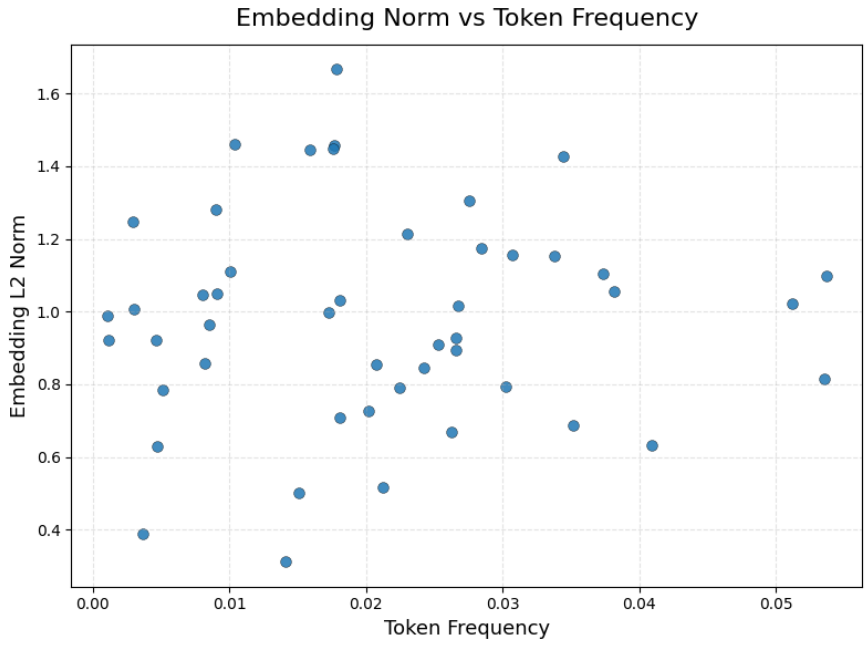


Figure 3.9: ℓ_2 norms of the 48 learned token embeddings, ordered by depth cluster. Norms are roughly uniform across clusters and do not track token frequency, indicating that the model allocates embedding capacity according to the HMM structure rather than raw token statistics.

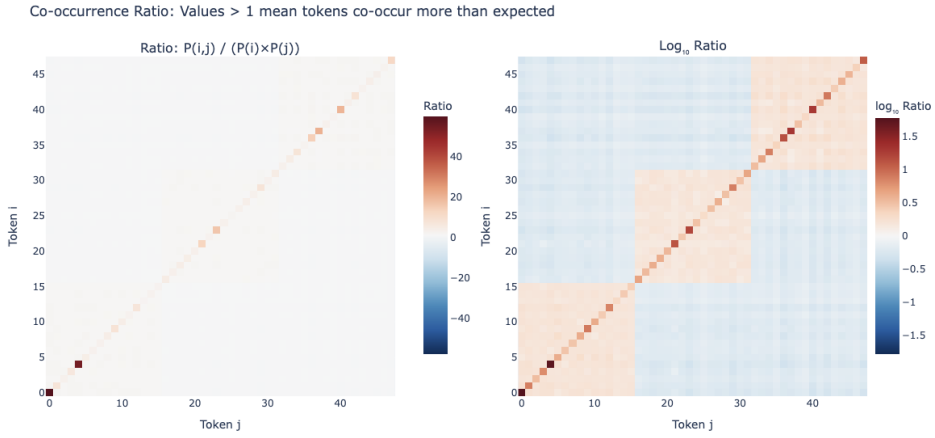


Figure 3.10: Comparison of the learned embedding Gram matrix $W_E^\top W_E$ with the log-PMI matrix computed from token co-occurrence statistics. The strong agreement between the two matrices supports the theoretical prediction $W_E^\top W_E \approx \log \mathrm{PMI}$ and explains why PMI-based (PPMI SVD) embedding initialisations perform well in the fixed-embedding experiments.

the transient dynamics may differ if the gradient signal quality depends on direction.

3.9.1 Setup

We extract the full validation loss curve $L(t)$ for each of the 146 training runs in the original grid ($n_{\text{layer}} \in \{1, 2, 4\}$, $n_{\text{embd}} \in \{4, 8, 16\}$, attention-only or full, with or without LayerNorm), comprising 86 forward and 60 reversed runs. Validation loss is recorded every 100 optimisation steps over 10 epochs ($\sim$483 data points per run).

For each run, we compute:

- τ_{50}: steps to reach 50% of total improvement ("half-life" of learning);
- τ_{90}: steps to reach 90% of total improvement;
- AUC: area under the excess loss curve $\int [L(t) - L_{\text{final}}]\, dt$ (total training cost);
- Epoch-1 loss: $L(t \approx 4834)$, reflecting early-training performance.

Metrics are averaged across seeds for configurations with multiple runs, then paired by architecture.

3.9.2 H5: Transient Dynamics Asymmetry

Table 3.4 summarises the paired deltas across 36 matched architecture configurations.

Metric	Mean Δ	Positive	t-test p	Wilcoxon p
$\Delta\tau_{50}$ (steps)	$+7.2 \pm 123.6$	8/36 (22%)	0.73	0.90
$\Delta\tau_{90}$ (steps)	-207.8 ± 841.3	16/36 (44%)	0.15	0.47
ΔAUC	$+23.4 \pm 255.4$	18/36 (50%)	0.59	0.87
Δ epoch-1 loss (nats)	-0.03 ± 0.10	15/36 (42%)	0.066	0.26

Table 3.4: Paired forward–reversed training dynamics metrics. Δ = reversed − forward; positive values indicate reversed is slower/higher. None of the metrics reach statistical significance.

Result: H5 is not supported. There is no significant difference in convergence speed between forward and reversed training. The marginally significant epoch-1 result ($p = 0.066$) has the *wrong sign*: reversed is slightly *faster* early in training, contrary to the prediction.

Figure 3.11 shows representative training curves. The forward and reversed loss curves are nearly indistinguishable across all architectures, with the largest visible differences attributable to seed variance rather than a systematic directional effect.

3.9.3 H6: Learning Phases

The early dynamics (Figure 3.12) show no evidence of a direction-dependent Phase II onset. Both forward and reversed curves decrease at the same rate during the first 1000 steps (Phase I unigram learning), consistent with the prediction that direction-independent marginal statistics dominate early training. However, the expected delayed Phase II onset for the reversed direction is not observed: the transition to context-dependent learning appears simultaneous for both directions.

Result: H6 is not supported. No phase-dependent directional asymmetry is detectable in the training dynamics.

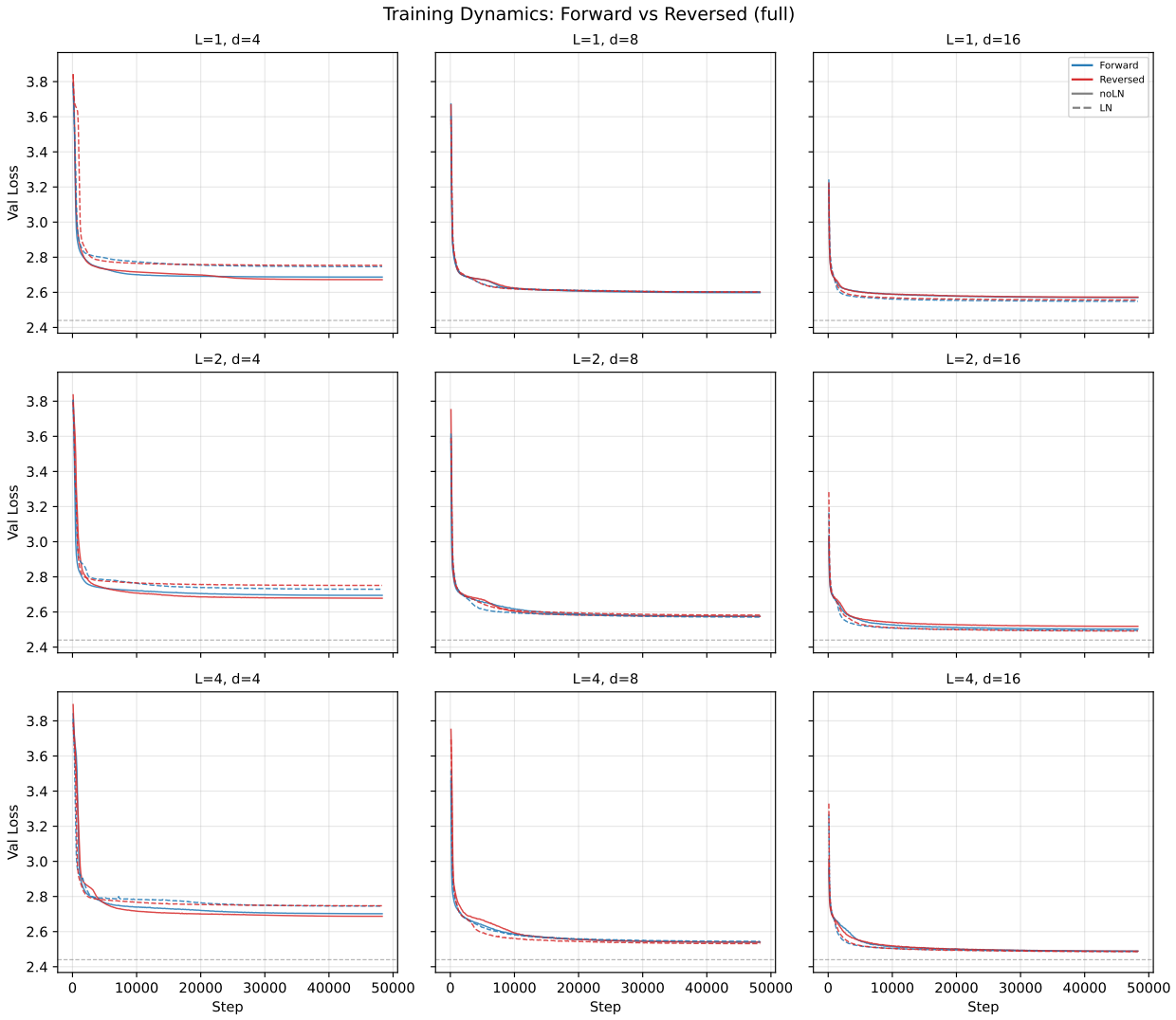


Figure 3.11: Validation loss curves for forward (blue) and reversed (red) training across the full architecture grid. Each panel shows one $(n_{\text{layer}}, n_{\text{embd}})$ combination for full (attn + MLP) models, with norm variants averaged. Shaded bands show ± 1 s.d. across seeds; the dashed grey line marks the entropy rate $h \approx 2.44$. The two directions are visually indistinguishable in most panels.

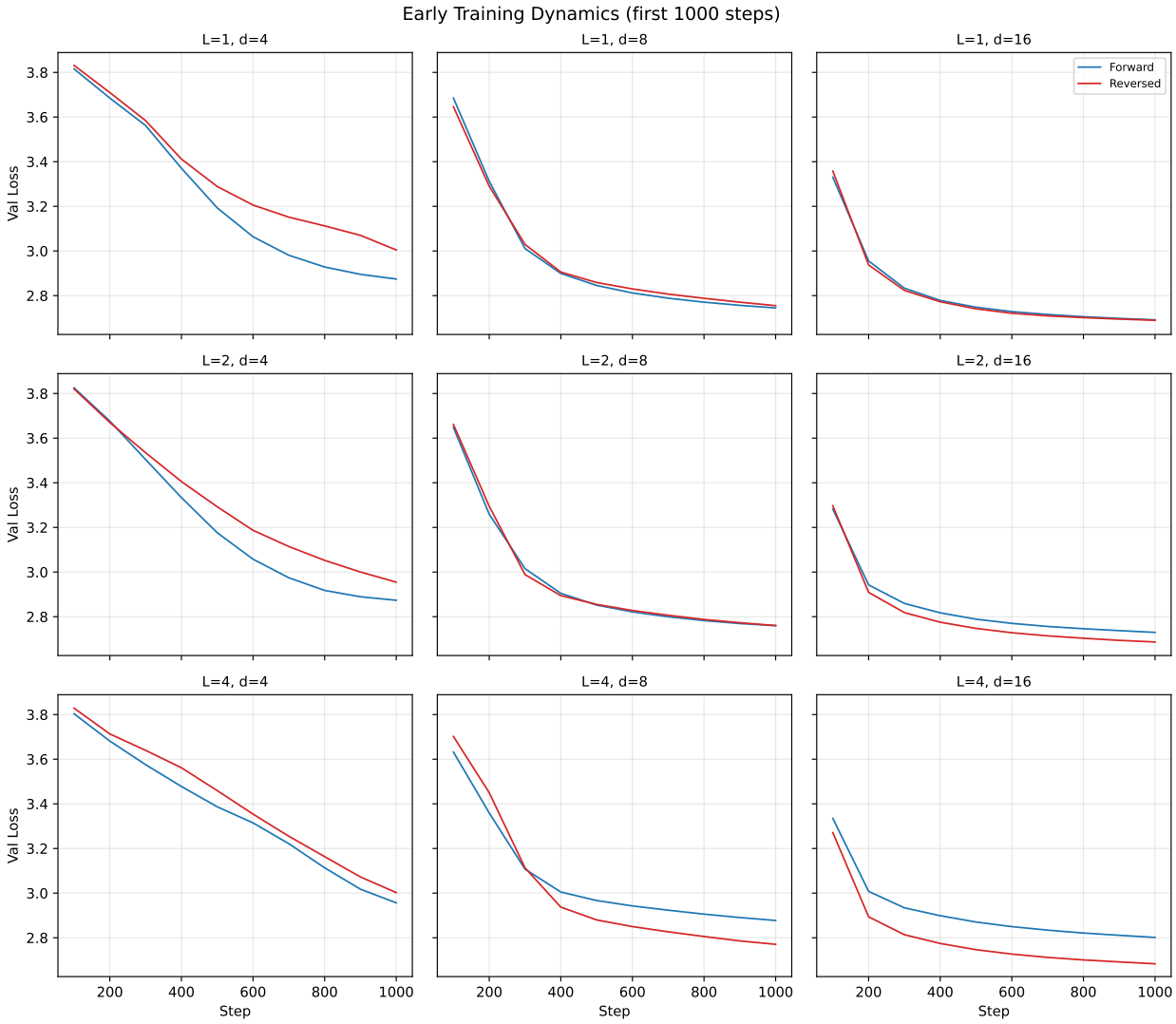


Figure 3.12: Early training dynamics (first 1000 steps) zoomed in. Forward and reversed curves are essentially overlapping during the unigram-learning phase, and no delayed Phase II onset is visible for the reversed direction.

3.9.4 H7: Architecture-Dependent Convergence Gap

Figure 3.13 shows the convergence speed gap as a function of model architecture and size.

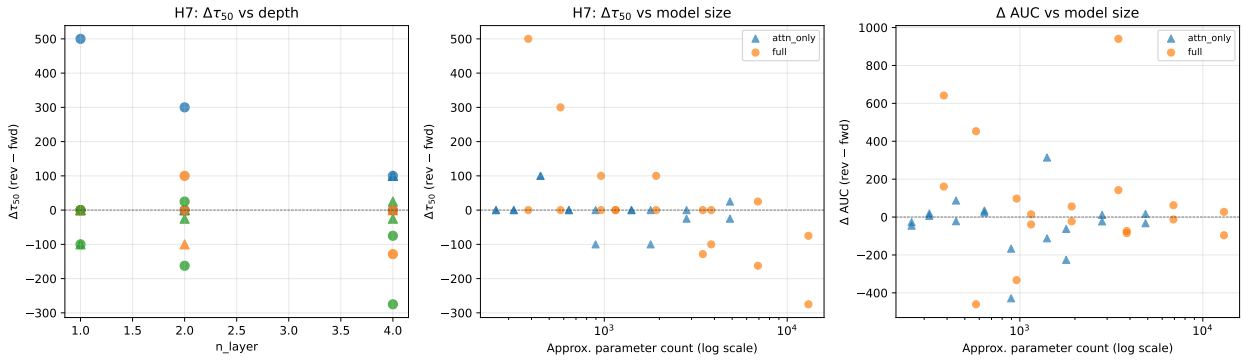


Figure 3.13: Architecture dependence of training dynamics gaps. Left: $\Delta\tau_{50}$ vs. number of layers. Centre: $\Delta\tau_{50}$ vs. parameter count. Right: ΔAUC vs. parameter count. Colours indicate n_{embd} (blue: 4, orange: 8, green: 16); triangles are attention-only, circles are full models.

The Pearson correlations are:

	$\text{Corr}(\cdot, \Delta\tau_{50})$	p-value
n_{layer}	-0.136	0.43
n_{embd}	-0.477	**0.003**
n_{params}	-0.484	**0.003**

Result: H7 is partially supported, but with the opposite sign. Larger models (in embedding dimension) show a significantly *smaller* $\Delta\tau_{50}$, meaning they slightly favour the reversed direction in convergence speed. The effect of depth alone (n_{layer}) is not significant.

3.9.5 Training Dynamics: Summary

The training dynamics analysis yields a clean null result for the central prediction (H5): forward and reversed training curves are statistically indistinguishable in convergence speed, excess training cost, and early-training performance. Combined with the Bayes-optimal gap vanishing at $k = 16$ ($\Delta_{\text{Bayes}} = +0.0006$ nats), this confirms that the context window is long enough to render the belief-state convergence asymmetry irrelevant—both to the optimal predictor and to the gradient-based optimisation process.

Additional figures (per-architecture bar charts, convergence scatter plots, epoch-1 vs. final gap comparison) are in `figures/training_dynamics/`.

3.10 Discussion and Open Questions

3.10.1 Summary of Findings

1. **H1 (entropy rate invariance) is confirmed.** Forward and reverse entropy rates agree to machine precision, as required by the stationarity of the process.

2. **H2 (finite-context asymmetry) is confirmed and quantified.** The reversed process has $\sim 6\times$ higher excess Bayes-optimal loss at $k = 1$, but the gap closes rapidly: by $k \approx 6$

both directions have converged to within Monte Carlo noise of $h_X \approx 2.44$ nats. At $k = 16$, $\Delta_{\text{Bayes}} = +0.0006$ nats.

3. **H3 (transformer learnability gap) is not supported.** The mean forward–reverse loss difference is $+0.0002 \pm 0.0086$ nats across 60 configurations, consistent with zero.

4. **H4 (architecture dependence) shows a weak negative correlation** between model size and Δ, opposite to the prediction.

5. **Belief-state geometry is spatially localised.** Activation ablations confirm that the model stores its belief-state representation in a low-dimensional subspace of the residual stream, with the fractal directions carrying substantially more predictive information than the orthogonal complement.

6. **Embedding geometry is consistent with PMI structure.** The learned embeddings satisfy an approximate $W_E^\top W_E \approx \log \text{PMI}$ relationship, connecting the empirical geometry to the theoretical optimum for linear models.

7. **H5 (transient dynamics asymmetry) is not supported.** Forward and reversed training curves are statistically indistinguishable in convergence speed ($\Delta\tau_{50}$: $p = 0.73$), excess training cost (ΔAUC: $p = 0.59$), and early-training loss ($p = 0.066$, wrong sign).

8. **H6 (learning phases) is not supported.** No direction-dependent Phase II onset is observed.

9. **H7 (architecture-dependent convergence gap) is partially supported with opposite sign.** Larger embedding dimension correlates with slightly faster reversed convergence ($r = -0.48$, $p = 0.003$), opposite to the prediction.

Unified interpretation. The Bayes-optimal gap at $k = 16$ ($\Delta_{\text{Bayes}} = +0.0006$ nats) provides the key to interpreting all null results. The belief-state convergence asymmetry (H2) is real: the reverse process needs $\sim 2\times$ more context to converge. However, at the training context length of $T = 16$, both directions have already converged to within 0.001 nats of the entropy rate. This means there is no information-theoretic basis for a learnability gap (H3), no reason for the optimisation landscape to differ (H5), and no phase-specific directional asymmetry (H6). The picture is complete: the context window is simply long enough to erase the asymmetry.

This suggests a clear experimental prediction: training on longer sequences ($T \gg 16$) is unnecessary to reveal a gap, but training on *shorter* sequences ($T \leq 4$, where Δ_{Bayes} is large) should produce a measurable forward–reverse difference.

3.10.2 Open Questions

- **Short-context regime:** Training with $T \leq 4$ (where $\Delta_{\text{Bayes}}(k) \approx 0.13$ nats at $k = 4$) should reveal the predicted forward–reverse gap. This is the most direct test of whether transformers are sensitive to belief-state convergence asymmetry when it is information-theoretically relevant.

- **N-gram baselines on reversed data:** Do n-gram models also show no forward–reverse gap at $T = 16$? If they do, the near-zero Δ is a general property of any fixed-context predictor and not specific to transformers.

- **Outlier investigation:** The `L4_d8_attn_only_noLN` configuration achieves $\Delta = -0.025$ nats. What mechanism makes an attention-only model find reversed sequences *easier*?
- **Ablations at full scale:** The activation ablations were performed on a single model. Repeating across the full architecture sweep would determine whether fractal-direction localisation is universal or architecture-dependent.
- **Fractal directions in the reverse model:** Does the reverse-trained transformer develop the same low-dimensional belief-state geometry? If so, what is the relationship between the fractal subspaces of the forward and reverse models?

Part III

Mechanistic Analysis

Chapter 4

Linear Probing: Code Review and Methodology

4.1 Purpose of This Document

This report is a code review of `old_src/pca.py`, which implements the pipeline for extracting fractal belief-state geometry from transformer residual streams. The review separates the *conceptual* content (what is being computed and why) from *engineering* concerns (how the code is structured, its scalability, and its correctness).

4.2 Conceptual Background

4.2.1 Belief States as an Iterated Function System

A Hidden Markov Model with N hidden states and vocabulary V defines a Bayesian belief update rule. After observing token x_t, the belief vector $\alpha_t \in \Delta^{N-1}$ (the probability simplex) evolves as

$$\alpha_{t+1} \;=\; \frac{E[x_t]^\top \alpha_t}{\mathbf{1}^\top E[x_t]^\top \alpha_t}, \tag{4.1}$$

where $E[x] \in \mathbb{R}^{N\times N}$ is the emission matrix for token x, with $E[x]_{ik} = P(\text{observe } x \text{ and transition } i \to k)$. Each token defines a contraction on the simplex; together the $|V|$ maps $\{f_x : x \in V\}$ form an **Iterated Function System** (IFS). The attractor of this IFS—the closure of all reachable belief states—is the **mixed-state presentation** (MSP). For generic HMMs this attractor is a fractal subset of the simplex [3, 4].

4.2.2 The Probing Question

Shai et al. [1] showed that transformers trained on next-token prediction over HMM-generated data learn an internal representation of the belief state that is recoverable via a *linear* map from the residual stream. Concretely, if $h_t^{(\ell)} \in \mathbb{R}^d$ denotes the residual-stream activation at layer ℓ and position t, then there exists a matrix $W \in \mathbb{R}^{N\times d}$ and bias $b \in \mathbb{R}^N$ such that

$$W h_t^{(\ell)} + b \;\approx\; \alpha_t. \tag{4.2}$$

The quality of this approximation, measured by R^2, quantifies how faithfully the transformer encodes the belief state at each layer.

4.2.3 Why "Fractal Directions"?

Because the belief simplex is $(N-1)$-dimensional (2D for Mess3's 3 states), only a low-dimensional subspace of the d-dimensional residual stream is needed to encode α_t. The principal components of this subspace are called the **fractal directions**: projecting activations onto them recovers the fractal geometry of the MSP. Ablating these directions destroys predictive performance far more than ablating the orthogonal complement (Section 5 of the time-reversal report), confirming their causal role.

4.3 What the Code Computes

The pipeline in `old_src/pca.py` has four stages:

1. **Model loading** (`load_model_from_weights`): Deserialise a trained transformer (either `HookedTransformerM` via TransformerLens or `MaskedHeadTransformerModel` via NNsight) from a checkpoint directory containing `config.yaml` and a `.pt` weights file.

2. **Activation extraction** (`extract_residual_stream`): Run a forward pass on a batch of observation sequences and record the post-layer residual-stream activations. Returns a tensor of shape `(num_layers, batch, seq_len, d_model)`.

3. **Linear probing** (`learn_affine_mapping`, `pca_layer_wise_actvs`, `pca_concat_actvs`): Fit ordinary least-squares regression from activations to ground-truth belief states. Two modes:

   - *Layer-wise*: One regression per layer, answering "at which layer does the belief representation crystallise?"
   - *Concatenated*: Stack all layers into a single feature vector, answering "is belief information distributed across layers?" (relevant for RRXOR, where a single layer's R^2 can be as low as 0.31).

4. **Visualisation** (`barycentric_to_cartesian_2d`, `plot_in_simplex`): Project 3D barycentric coordinates to an equilateral triangle for plotting.

The entry point `run_pca_and_save_results` orchestrates all four stages over one or more model directories, with optional MPI parallelism.

4.4 Conceptual Critique

4.4.1 The Name "PCA" Is a Misnomer

Despite the filename, **no PCA is performed on the activations**. The code fits a linear regression (ordinary least squares via `sklearn.LinearRegression`) from activations to belief states. PCA and linear regression are related—PCA finds the directions of maximum variance in the *input* space, while regression finds the directions that maximally predict a *target*—but they are distinct operations. The code should be renamed (e.g., `linear_probe.py` or `belief_probe.py`) to avoid confusion.

The `sklearn.decomposition.PCA` import on line 12 is in fact *unused*.

4.4.2 Appropriateness of a Linear Probe

The choice of a linear (affine) probe is well motivated: it tests whether the belief state is linearly decodable from the residual stream. A nonlinear probe (e.g., an MLP) would achieve higher R^2 but would not distinguish "the information is linearly accessible" from "the information exists somewhere in the activations and can be extracted with sufficient computation." The linear probe is the standard tool in the mechanistic-interpretability literature for this reason [1, 5].

One subtlety: the probe includes a bias term (b in the affine map), which is important because belief states are constrained to the simplex ($\sum_i \alpha_i = 1$) while activations are not. The bias absorbs the mean of the belief distribution.

4.4.3 Last-Position vs. All-Position Analysis

The `use_last_pos` flag (default `True`) restricts the analysis to the final sequence position. This is the natural choice for autoregressive models: the last-position activation has seen the full context and should encode the most refined belief state. When `use_last_pos=False`, the code reshapes all positions into a single regression, mixing different context lengths. This is a weaker test—short-context positions will have diffuse beliefs (close to the stationary distribution) and inflate R^2 by providing "easy" data points near the simplex centre.

4.4.4 Metric Interpretation

Three metrics are reported:

- **MSE**: Mean squared error between predicted and true belief states, averaged over all simplex dimensions and data points.
- R^2: Coefficient of determination. Values above 0.99 indicate near-perfect linear decodability.
- **Baseline MSE**: The MSE achieved by predicting the mean belief state for all inputs. The ratio MSE/Baseline MSE $= 1 - R^2$ relates the three metrics.

Reporting all three is mildly redundant (any two determine the third) but provides useful sanity checks.

4.4.5 Missing: Statistical Rigour

The code does not perform train/test splitting of the probing data. All 10^5 data points are used for both fitting and evaluation. With $d = 8$ (or even $d = 32$ for concatenated features) and 10^5 samples, overfitting is unlikely in practice, but a held-out split would strengthen the claims. More importantly, no confidence intervals or significance tests are reported for R^2 differences across layers.

4.5 Engineering Critique

4.5.1 Strengths

- **MPI parallelism.** The `run_pca_and_save_results` function distributes model directories across MPI ranks with round-robin GPU assignment. Rank 0 generates belief states once and broadcasts them, avoiding redundant computation. Error handling is robust: each directory is processed in a try/except block with per-rank summaries.

- **Automatic device selection.** The `get_device()` helper correctly prioritises CUDA → MPS → CPU.
- **Skip-if-done logic.** Already-processed directories (containing `pca_data.npz`) are skipped, enabling restartable batch runs.
- **Comprehensive serialisation.** Regressors (pickle), activations/predictions (compressed npz), and metrics (CSV) are all saved, enabling downstream analysis without re-running the pipeline.

4.5.2 Issues

1. **Unused imports.** `PCA` (line 12), `math` (line 14), `F` (line 7), `transformer_lens` (line 22), and `NNsight` (line 21) are imported but never used in the module's own functions. The `NNsight` import triggers a side effect (registering the `.trace()` context manager on `nn.Module`), which is required for the `MaskedHeadTransformerModel` branch of `extract_residual_stream`. This implicit dependency should be documented.
2. **Model lookup via `globals()`.** Line 54 resolves the model class by name from `globals()`, which depends on the specific imports at the top of the file (`HookedTransformerModel`, `MaskedHeadTransformerModel`). This is fragile: adding a new model class requires editing the import block. A registry pattern (as in `src/utils.py`'s `PROCESS_REGISTRY`) would be more maintainable.
3. **Hard-coded HMM process.** The entry point (line 518 onward) hard-codes `Mess3Proc()` and the filename `final_model_mess3.pt`. The HMM process should be read from the model's `config.yaml` to support other processes (Z1R, RRXOR, CylinderGraphHMM) without code changes.
4. **Memory scaling.** With `batch_size=100 000` and `seq_length=16`, the belief-state tensor is shape $(10^5, 16, N)$ and the activation tensor is $(L, 10^5, 16, d)$. For a 4-layer model with $d = 8$, this is only $\sim$50 MB. But for the CylinderGraphHMM with $N = 18$, $d = 64$, $L = 4$, the activation tensor alone is $\sim$3.3 GB. The code loads everything into memory simultaneously. For larger models, a streaming or chunked approach would be needed.
5. **Belief states on wrong device.** Line 611 creates `t.tensor(truncated_beliefs, device=device)`, but `truncated_beliefs` is a NumPy array and the belief states are only used by `learn_affine_mapping`, which immediately calls `.cpu().numpy()`. The GPU round-trip is unnecessary.
6. **Redundant computation in `pca_layer_wise_actvs`.** Lines 255–263 recompute the per-layer activation slicing that was already done in the main loop (lines 223–231), solely to save the data. This duplicates the slicing logic and risks inconsistency.
7. **No `run_id` in layer-wise output filenames.** The layer-wise functions save to fixed names (`pca_data.npz`, `pca_metrics.csv`, `regressors.pkl`) without incorporating a run identifier, unlike the concatenated variant which uses `run_id` prefixes. This makes it impossible to store multiple analyses in the same directory.

4.6 Refactored Implementation

The monolithic `old_src/pca.py` has been decomposed into five modules under `src/`, each with a single responsibility and a corresponding test suite under `tests/`.

4.6.1 Architecture

`src/linear_probe.py` Core probing logic (pure NumPy/sklearn, no I/O, no Torch). Exposes a `ProbeResult` dataclass and three entry points: `fit_probe_with_split`, `probe_layer_wise`, and `probe_concatenated`. All functions take a `test_fraction` and `seed` argument to produce reproducible train/test splits.

`src/activation_extraction.py` Model loading via `initialize_transformer_from_yaml` (from `src/utils.py`) and residual-stream extraction via TransformerLens's `run_with_cache`. Supports chunked inference (`batch_size` parameter) for large datasets. Auto-detects the best checkpoint file (priority: `best_model.pt` > `latest.pt` > highest-epoch `checkpoint_epoch_*.pt`). The HMM process is read from `config.yaml` via `create_process_from_dict`, eliminating the hard-coded `Mess3Proc` dependency.

`src/visualisation.py` Simplex plotting (matplotlib). Ported from the original code with added `ax` parameter for subplot support and configurable `vertex_labels`.

`src/probe_io.py` Serialisation: `save_probe_results` writes a metrics CSV (with train and test columns), pickled regressors, and optionally compressed NPZ data. All filenames use a consistent `run_id` prefix. `load_probe_results` is the inverse.

`src/run_linear_probe.py` CLI entry point. Supports single-model (`--model-dir`) and batch (`--model-dirs`) modes with optional MPI parallelism. In batch mode, Rank 0 generates shared sequences and belief states, then broadcasts to all ranks—the same pattern as the original code.

4.6.2 Data Flow

The pipeline proceeds as follows:

1. Read `config.yaml` from the model directory to determine the HMM process and model architecture.
2. Generate observation sequences and compute ground-truth belief states via `HMM.mixed_state_presentation`.
3. Load the trained `HookedTransformer` and extract post-layer residual-stream activations with `extract_residual_stream`.
4. Reshape activations and beliefs via `prepare_probe_data` (last-position or all-position).
5. Fit linear probes via `probe_layer_wise` and/or `probe_concatenated`, each with a proper train/test split.
6. Save results via `save_probe_results`.

4.6.3 Output Format

Running the CLI produces a `probe_results/` directory inside the model directory (or at a custom `--output-dir`) with files:

- `layerwise_metrics.csv` — one row per layer with columns: `layer`, `train_mse`, `test_mse`, `train_r2`, `test_r2`, `baseline_mse`.

- `concat_metrics.csv` — single row for the concatenated probe.
- `*_regressors.pkl` — pickled `LinearRegression` objects.
- `*_data.npz` — (optional, with `--save-data`) train/test predictions.

4.7 Usage Guide

4.7.1 Single Model

```
python src/run_linear_probe.py \
    --model-dir models/cylinderhmm/20250301_run1/
```

4.7.2 Batch Processing

```
python src/run_linear_probe.py \
    --model-dirs models/cylinderhmm/*/
```

4.7.3 MPI Parallelism

```
mpirun -n 4 python src/run_linear_probe.py \
    --model-dirs models/cylinderhmm/*/ --use-mpi
```

4.7.4 Loading Results in a Notebook

```
from probe_io import load_probe_results
results = load_probe_results("models/.../probe_results",
                             run_id="layerwise")
print(results["metrics"])
```

4.8 Summary of Recommendations

References

Category	Priority	Recommendation	Status
Conceptual	High	Rename module from `pca.py` to `linear_probe.py` (Section 4.4.1).	**Resolved**
Conceptual	Medium	Add train/test split to probing evaluation.	**Resolved**
Conceptual	Low	Consider nonlinear probe as comparison baseline.	Open
Engineering	High	Read HMM process from `config.yaml` instead of hard-coding `Mess3Proc`.	**Resolved**
Engineering	Medium	Replace `globals()` model lookup with registry.	**Resolved**
Engineering	Medium	Remove unused imports; drop NNsight dependency.	**Resolved**
Engineering	Low	Eliminate GPU round-trip for belief states.	**Resolved**
Engineering	Low	Unify `run_id` naming convention.	**Resolved**

Table 4.1: Summary of recommendations with resolution status after refactoring.

Bibliography

[1] A. Shai, S. Marzen, L. Teixeira, A. Oldenziel, and P. Riechers, "Transformers represent belief state geometry in their residual stream," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. `arXiv:2405.15943`.

[2] A. Shai, P. Riechers, et al., "Constrained belief updates explain geometric structures in transformer representations," in *International Conference on Machine Learning (ICML)*, 2025. `arXiv:2502.01954`.

[3] P. Jurgens and J. P. Crutchfield, "Shannon entropy rate of hidden Markov processes," *Journal of Statistical Physics*, vol. 183, 2021.

[4] J. P. Crutchfield and K. Young, "Inferring statistical complexity," *Physical Review Letters*, vol. 63, pp. 105–108, 1989.

[5] G. Alain and Y. Bengio, "Understanding intermediate layers using linear classifier probes," in *ICLR Workshop*, 2017.

[6] N. Aggarwal, S. R. Dalal, and V. Misra, "The Bayesian geometry of transformer attention," `arXiv:2512.22471`, 2025.

Chapter 5

Iterative Probing and Counterfactual Importance

5.1 Introduction

Previous probing experiments on transformers trained on Hidden Markov Model (HMM) data have established that the residual stream encodes a high-fidelity linear image of the Bayesian belief state (the posterior distribution over hidden states given the observed token history). However, initial ablation experiments—in which the three SVD directions of a single linear probe's coefficient matrix were mean-ablated one at a time—produced KL divergences only marginally larger than those from ablating random directions. This raised the possibility that the linearly decodable belief state is not the representation the model actually *uses* for prediction, or that the causal signal is distributed across more directions than the three extracted from a single probe.

These observations motivate two sharper questions: *how many independent copies* of the belief state does the network store in its residual stream, and *which copies matter* for the model's actual next-token predictions?

These questions bear on two distinct aspects of the representation:

1. **Redundancy.** A 64-dimensional residual stream need only devote 2 dimensions to represent the Mess3 belief state (which lives on a 2-simplex). If the network stores multiple independent copies, this tells us something about how superposition and redundancy interact in small transformers—and whether the network "wastes" capacity or exploits it for robustness.

2. **Causal relevance.** A subspace may be linearly decodable without being causally used by downstream layers. Counterfactual ablation (replacing a subspace's projection with its mean) tests whether the model's output *changes* when that information is erased, distinguishing representation from mere correlation.

This report describes the *iterative probing* algorithm and its application to a 4-layer, 64-dimensional, single-head transformer trained on the Mess3 process.

5.2 Model and Process

The Mess3 process (`src/hmm.py`, class `Mess3Proc`) is a 3-state, 3-token HMM. The model is a 4-layer HookedTransformer with $d_{\text{model}} = 64$, a single attention head ($d_{\text{head}} = 64$), MLP width 256,

ReLU activations, and LayerNorm. It was trained on 10M tokens with context length $n_{\text{ctx}} = 10$. The geometric structure of belief-state representations in transformers trained on HMMs has been studied by Shai et al. [1]. The model directory is:

`models/mess3/20260306_150920_L4_d64_H1_full_LN/`

The belief state for Mess3 has 3 components summing to 1, so it lives on a 2-dimensional simplex embedded in $\mathbb{R}^3$. A linear probe mapping $\mathbb{R}^{64} \to \mathbb{R}^3$ therefore has a coefficient matrix of shape 3×64 whose row space is generically 2-dimensional (or at most 3, depending on how the intercept interacts with the sum-to-one constraint).

5.3 Algorithm

5.3.1 Iterative Probing

The core idea is to repeatedly extract and remove the best linear predictor of the belief state from the activations, counting how many times this can be done before the residual signal drops below a significance threshold.

For each layer ℓ:

1. Let $X^{(0)} \in \mathbb{R}^{n \times 64}$ be the last-position residual-stream activations.
2. At iteration k:
   (a) Fit a linear probe: $\hat{y} = X^{(k)} W^\top + b$, where $W \in \mathbb{R}^{3 \times 64}$.
   (b) Evaluate test R^2. If $R^2 < \tau$ (default $\tau = 0.05$), stop.
   (c) Compute the SVD of $W = U \Sigma V^\top$. Retain rows of $V^\top$ corresponding to singular values above a relative threshold ($\sigma_i / \sigma_0 > 10^{-3}$). Call the retained rows $V_k \in \mathbb{R}^{r_k \times 64}$, where r_k is the effective rank.
   (d) Project out: $X^{(k+1)} = X^{(k)} - X^{(k)} V_k^\top V_k$.
3. The number of iterations with $R^2 \geq \tau$ is the number of "independent copies."

This is implemented in `src/iterative_probe.py`, with the main entry points being `iterative_probe_single_l` and `iterative_probe_all_layers()`. The function `extract_subspace()` performs the SVD and rank thresholding; `project_out_subspace()` handles the orthogonal projection. The algorithm reuses `fit_probe_with_split()` from `src/linear_probe.py` at each iteration to ensure a consistent 80/20 train/test split.

Why R^2 and not MSE? MSE depends on the scale of the targets, while R^2 is normalised. Because the projected activations have shrinking variance with each iteration, MSE alone could be misleading; R^2 measures the fraction of target variance explained regardless of the activation scale.

5.3.2 Counterfactual Importance via Mean Ablation

For each layer ℓ and each iteration-k subspace V_k:

1. Run the model cleanly to obtain reference log-probabilities $\log p_{\text{clean}}$ at the last token position.

2. Intervene on the residual stream at layer ℓ using a `run_with_hooks` forward hook: project the activations onto V_k, replace the projection with its batch mean, and add back the residual. Formally, if a is the activation vector:

$$a' = a - V_k^\top V_k\, a + \mathbb{E}_{\text{batch}}[V_k^\top V_k\, a]. \tag{5.1}$$

3. Compute the KL divergence: $D_{\text{KL}}(p_{\text{clean}} \| p_{\text{ablated}})$, which measures how much the model's output distribution shifts when this subspace is erased.

4. Also compute the cross-entropy change ΔCE as a secondary metric.

This is implemented in `src/run_iterative_probe.py`, functions `ablate_subspace()` and `compute_counterfac` The ablation generalises the single-direction ablation in `archive/old_src/ablation.py` to arbitrary-dimensional subspaces via the einsum-based projection `t.einsum("bsd,rd->bsr", actvs, subspace)`.

We additionally ablate each individual SVD direction within a subspace and the cumulative subspace (all iterations combined) to obtain finer-grained attribution.

5.4 Results

The experiment was run with 10,000 sequences of length 10, using the last token position. The algorithm was run for all 32 possible iterations (exhausting the full 64-dimensional space with rank-2 subspaces), with no R^2 stopping threshold.

Note on position selection. We probe only the last sequence position, which gives 10,000 samples. Using all positions would increase the sample count 10-fold to 100,000, potentially improving probe stability and revealing subtler structure at early positions where the belief state has not yet converged. However, last-position probing is sufficient for the present analysis and avoids conflating position-dependent effects (e.g., the belief state is less well-determined at early positions) with the redundancy structure we aim to measure.

5.4.1 Iterative Probing: Many Redundant Copies

Figure 5.1 shows the full trajectory of R^2 across all 32 iterations. Each iteration extracts a rank-2 subspace (consistent with the 2D simplex geometry of the Mess3 belief state), so 32 iterations exhaust all $32 \times 2 = 64$ dimensions of the residual stream.

The R^2 curve reveals three distinct regimes:

1. **Plateau** (iterations 0–20, 0–40 dims): R^2 remains above 0.93 at all layers, declining only gradually. The belief state is linearly decodable from at least 20 mutually orthogonal 2D subspaces.

2. **Transition** (iterations 20–28, 40–56 dims): R^2 begins to fall more steeply, particularly at later layers. By iteration 27 (54 dims removed), layer 3 drops to $R^2 \approx 0.69$.

3. **Collapse** (iterations 28–31, 56–64 dims): R^2 drops sharply as the remaining dimensions are insufficient to represent the belief state. The final iteration yields $R^2 \approx 0.19$–0.24, consistent with near-chance prediction from 2 remaining dimensions.

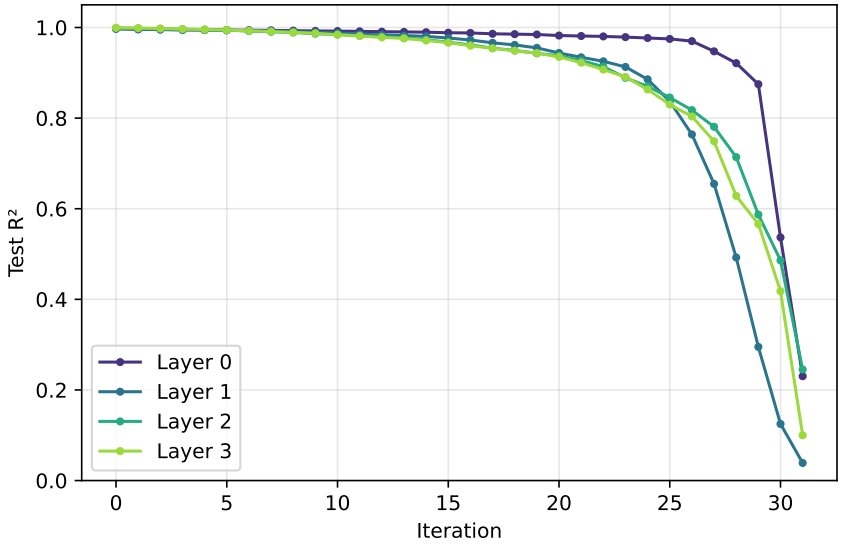


Figure 5.1: Test R^2 as a function of iteration for each layer across all 32 iterations. R^2 remains above 0.95 through ∼25 iterations before collapsing as the residual stream is exhausted.

The degradation is not uniform across layers. Layer 0 retains the highest R^2 longest (0.93 at iteration 28), while layer 3 degrades fastest (0.62 at iteration 28). This suggests that the belief-state signal is spread more uniformly across dimensions at layer 0, while at later layers it is more concentrated in a smaller subspace.

Figure 5.2 shows the corresponding MSE growth. Later layers start with lower MSE (layer 3 begins at 1.3×10^{-4}, vs. 4.4×10^{-4} for layer 0), reflecting a more precise belief-state encoding deeper in the network. The MSE growth is approximately linear through the first ∼40 dimensions removed, suggesting that the redundant copies contribute roughly equally to probe accuracy in this regime. Beyond 40 dimensions, the MSE curve steepens superlinearly, and by 64 dimensions (all projected out) the MSE reaches 0.13–0.16, comparable to predicting the marginal mean.

5.4.2 Counterfactual Importance: A Strong Hierarchy

Figure 5.3 reveals a strikingly different picture from the probing results. While all 32 subspaces at each layer are *decodable* (with $R^2 > 0.19$ even at the final iteration), they are far from equally *important*:

- **Layer 3, iteration 0** produces the largest KL shift by a wide margin: $\text{KL} = 0.040 \pm 0.027$. The second iteration at layer 3 drops by 12× to $\text{KL} = 0.003$. Subsequent iterations fluctuate around 10^{-4}–10^{-3}, with no systematic pattern.
- **Layer 2** shows a similar but weaker pattern: iteration 0 gives $\text{KL} = 0.009$, iteration 1 gives 0.002, then the signal drops to $\sim 10^{-4}$.
- **Layer 0** subspaces are almost entirely causally inert: individual iterations give $\text{KL} \sim 10^{-5}$–10^{-4}, though the later iterations (25–31) show slightly elevated KL ($\sim 10^{-3}$), likely because ablating the last few dimensions begins to disrupt the residual stream's overall structure regardless of belief-state content.

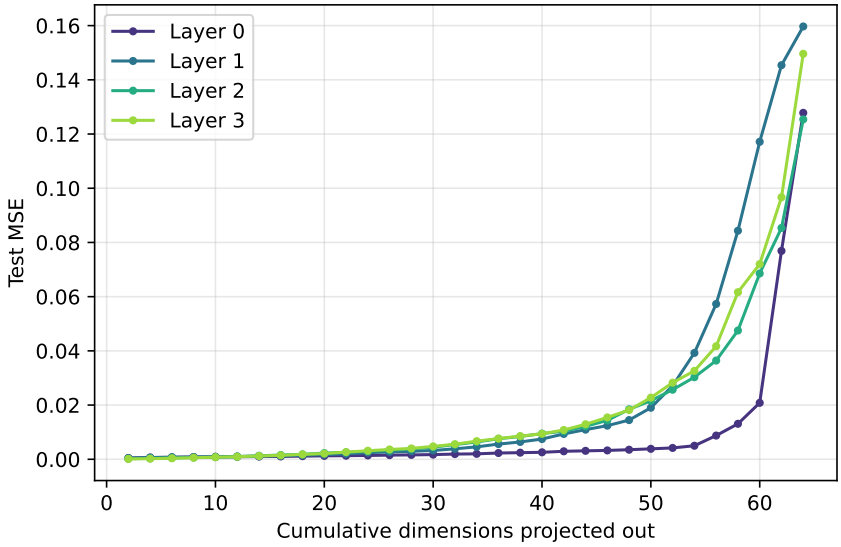


Figure 5.2: Test MSE vs. the number of dimensions projected out. MSE grows slowly through ∼40 dims, then accelerates as the remaining subspace can no longer support the belief state.

- The causal importance follows a clear **layer hierarchy**: layer 3 ≫ layer 2 > layer 1 ≫ layer 0.

The per-iteration ablations (Figure 5.3) show each subspace's importance *in isolation*. A complementary question is: how does the causal impact accumulate as we ablate progressively more subspaces? Figure 5.4 answers this by jointly ablating subspaces 0 through k for each $k = 0, 1, \ldots, 31$, covering 2 to 64 dimensions.

The cumulative-progressive curves reveal several features:

- **Layer 3** rises steeply in the first few iterations (driven by the dominant iteration-0 subspace), then continues to grow steadily as additional subspaces contribute incrementally. The final value (all 64 dims ablated) represents a complete erasure of all linear structure in the residual stream.
- **Layer 0** remains nearly flat across all 64 dimensions, confirming that belief-state-correlated directions at layer 0 are collectively as well as individually inert.
- At all layers, the curve is concave: the *marginal* contribution of each additional subspace diminishes. The first 2 dimensions account for the majority of the causal signal; subsequent subspaces add only small increments.
- The layer ordering is preserved throughout the entire range: layer 3 > layer 2 > layer 1 > layer 0 at every value of k.

Figure 5.5 juxtaposes the two perspectives. The flat R^2 curves and the steeply declining KL bars make the dissociation visually clear: decodability does not imply causal relevance.

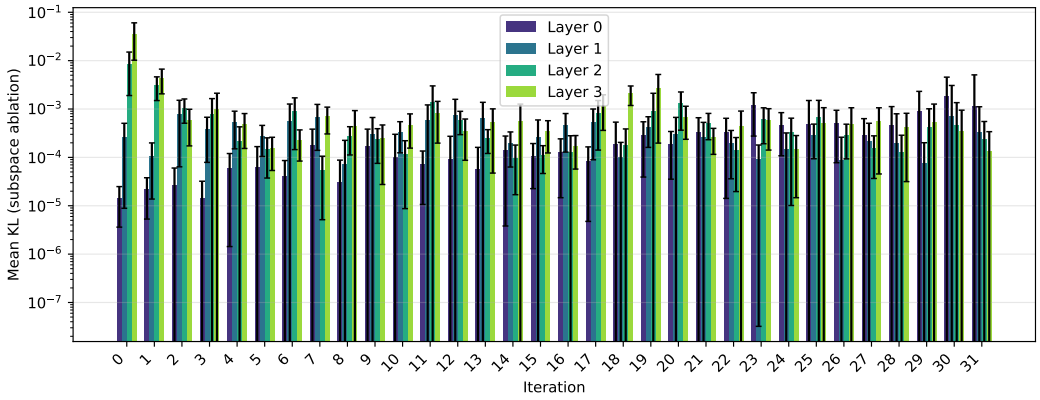


Figure 5.3: Mean KL divergence from ablating each iteration's subspace, broken down by layer. Note the log scale. Iteration 0 at layer 3 dominates by an order of magnitude.

5.5 Training Dynamics: Emergence of Redundant Copies

The redundancy documented above characterises the *final* model. A natural follow-up question is: does the multi-copy structure emerge during training, or is it already present at random initialisation?

To answer this, we ran the iterative probing algorithm (32 iterations, no R^2 cutoff) on four snapshots of the same architecture: the randomly initialised model (step 0), an early checkpoint (step 100), an intermediate checkpoint (step 2000), and the fully trained best model (∼46k steps). All four evaluations used the same 10,000 held-out sequences and belief states for comparability.

Figure 5.6 reveals that the redundant copy structure is **genuinely learned**, not an artefact of random initialisation. Several observations stand out:

1. **Random init baseline.** Even at initialisation, the belief state is partially decodable ($R^2 \approx 0.93$ at iteration 0). This is expected: random linear projections of a structured signal retain significant linear predictability, especially in a 64-dimensional space projecting a 2D target. However, R^2 decays *steadily* with each iteration—there is no plateau, and all layers are nearly indistinguishable.

2. **Early training (step 100).** The first-iteration R^2 increases slightly ($\sim 0.93 \rightarrow 0.94$), and a mild layer ordering begins to appear, but the decay profile remains qualitatively similar to random init.

3. **Intermediate training (step 2000).** The first-iteration R^2 rises further to ∼0.96, and a shallow plateau begins to form over the first 10–15 iterations. Layer differentiation becomes more pronounced, with deeper layers showing faster decay after the plateau—the opposite of the best-model pattern.

4. **Fully trained (best model).** A qualitative phase transition: R^2 stays above 0.98 for ∼20 iterations, forming the extended plateau documented in Section 5.4. Layer 0 now retains the highest R^2 longest, while layer 3 degrades first—a reversal of the random-init ordering.

The contrast between random init and the trained model is most informative. At random init, the belief-state signal is "smeared" uniformly across all 64 dimensions by the random weight matrices; the steady R^2 decay simply reflects the decreasing number of available dimensions. Training

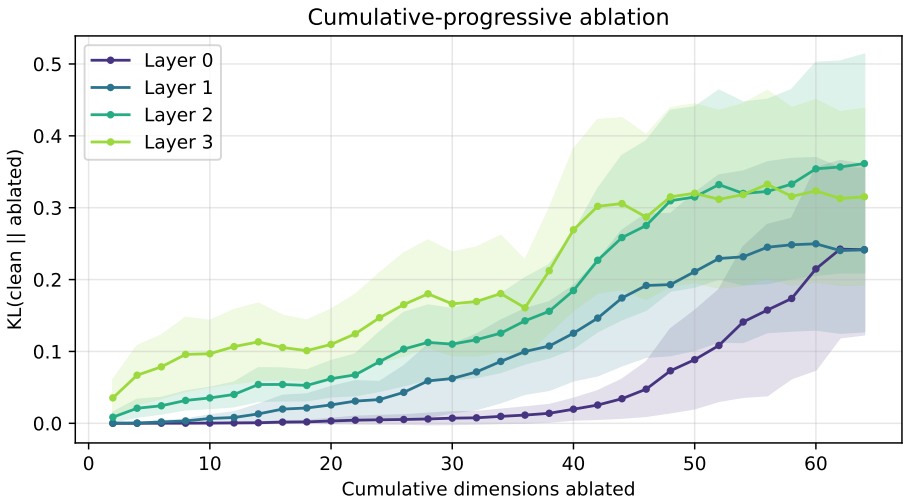


Figure 5.4: Cumulative-progressive ablation: KL divergence when subspaces $0, \ldots, k$ are jointly mean-ablated, as a function of the number of ablated dimensions $2(k{+}1)$. Shaded regions show ± 1 standard deviation across sequences.

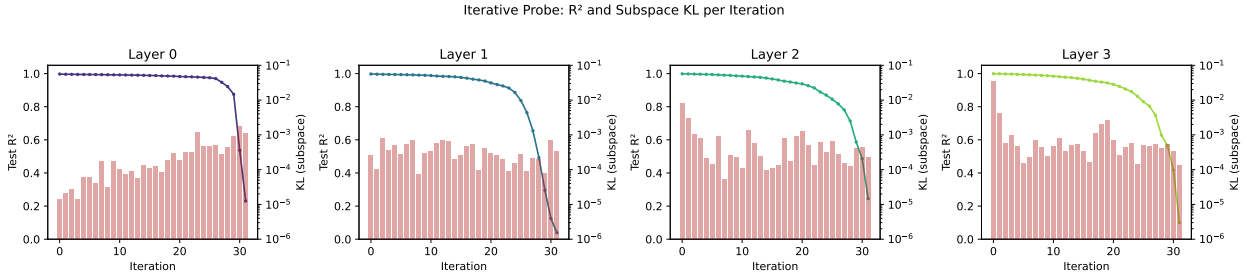


Figure 5.5: Dual-axis plots showing decodability (R^2, line) and causal importance of individual subspaces (KL, red bars) together for each layer. The contrast between flat R^2 and sharply decaying KL is most pronounced at layer 3.

reorganises the residual stream so that belief-state information is encoded with much higher fidelity and maintained across many more orthogonal projections.

The layer ordering reversal is also notable. At random init, all layers behave identically (since each is just a different random projection). In the trained model, the layer hierarchy (layer 0 most robust, layer 3 degrading first in terms of redundancy) reflects the functional specialisation induced by training. Taken together with the ablation results of Section 5.4, this suggests that training concentrates the *causally important* belief-state signal into a small subspace at layer 3, while distributing redundant (but non-causal) copies more broadly at earlier layers.

5.5.1 Random-Init Decodability Is a Generic Reservoir Effect

The high $R^2 \approx 0.93$ at random initialisation might appear surprising, but it is a well-understood consequence of random feature maps. A randomly initialised transformer maps each input sequence to a 64-dimensional activation vector via a deterministic (albeit arbitrary) nonlinear function. Since the belief state is also a deterministic function of the same input, a linear probe can exploit the

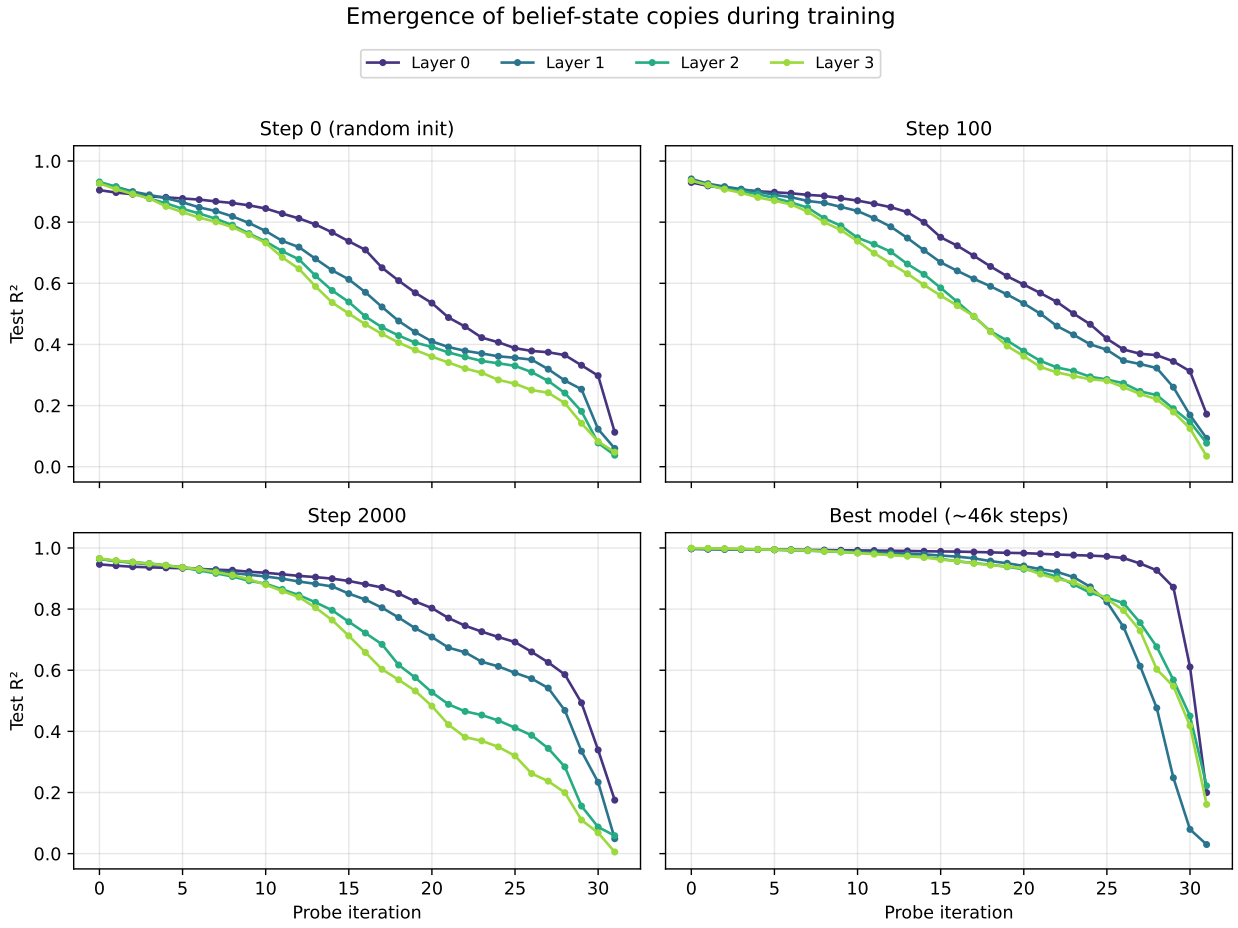


Figure 5.6: R^2 vs. probe iteration at four training stages. At random initialisation (top left), R^2 decays steadily from ∼0.93, with all layers behaving similarly. By step 100 (top right), the curves are slightly lifted but still show no plateau. At step 2000 (bottom left), the trained structure is emerging but the extended plateau is absent. The best model (bottom right) shows a qualitatively different regime: R^2 remains above 0.98 through ∼20 iterations before collapsing.

random network's activations as a *reservoir*—a high-dimensional nonlinear feature space from which smooth, low-dimensional target functions are linearly decodable [2, 3].

We verified this with three control experiments:

1. **Input-dependence is essential.** Replacing the random-init activations with IID Gaussian vectors (same shape, no input dependence) or shuffling the activation–sample correspondence both yield $R^2 \approx 0$. The high R^2 at random init is therefore not an overfitting artefact but requires the features to be a deterministic function of the input.

2. **Belief state is not special.** We probed the *same* random-init activations for eight different target functions of the Mess3 input sequences: the true belief state, token counts, last-token identity, bigram counts, position-weighted averages, a trigram hash, modular arithmetic, and belief states computed under a wrong HMM model. All smooth, low-dimensional targets yielded high R^2 (0.51–0.99), while the highly nonlinear modular-sum target gave $R^2 \approx 0.02$. The true belief state ($R^2 = 0.96$) fell squarely in the middle of the distribution.

3. **Wrong-model belief states are equally decodable.** We generated 20 random emission matrices (with 2–5 hidden states each) and computed Bayesian belief updates on the same Mess3 token sequences using these completely wrong HMMs. The resulting R^2 values had mean 0.96 ± 0.03 (range 0.87–0.99), indistinguishable from the true belief state's $R^2 = 0.96$.

Implications for baselines in probing studies. These results bear on the validation methodology of Shai et al. [1], who verify that their belief-state probes are non-trivial by shuffling belief-state labels (breaking the input–target correspondence) and showing that the probe collapses. This control is analogous to our IID/shuffled conditions and rules out projection artefacts, but it does not address the random-init baseline: a randomly initialised network already achieves $R^2 \approx 0.96$ for *any* smooth function of the input, including the belief state. A fairer baseline would compare trained-model R^2 against random-init R^2, which reveals that the true contribution of training is the refinement of copy quality ($0.93 \to 0.99$) and the creation of a persistent multi-copy plateau, rather than the emergence of decodability *per se*.

5.6 Unembedding Decomposition: How W_U Reads Belief-State Subspaces

The iterative probing analysis established that the residual stream contains 25+ redundant copies of the belief state, of which only subspace 0 at layer 3 is causally important. We now ask: *how does the unembedding matrix W_U convert belief-state information into next-token logits?*

The unembedding matrix $W_U \in \mathbb{R}^{64\times 3}$ maps the post-LayerNorm residual stream to logits over the 3-token Mess3 vocabulary. Since W_U has rank at most 3, it can read from at most ∼1–2 of the 2D belief-state subspaces. The decomposition $W_U = \sum_k V_k^\top (V_k W_U)$ is always valid by Parseval's identity (where $\{V_k\}$ are the orthogonal probe subspaces), but most terms must be negligible.

Post-LN probing. Because `ln_final` sits between the residual stream and W_U, we first verify that belief-state information survives the final LayerNorm. We run the same iterative probing algorithm on the *post-LN* activations (extracted via `cache["ln_final.hook_normalized"][:, -1, :]`).

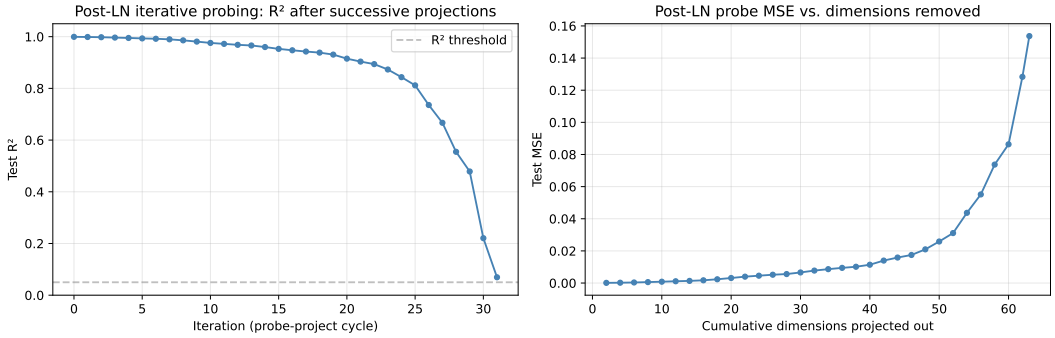


Figure 5.7: Iterative probing on post-LN activations. **Left:** R^2 vs. probe iteration shows the same extended plateau as pre-LN probing, confirming that belief-state information persists through the final LayerNorm. **Right:** MSE vs. cumulative dimensions removed grows slowly through ∼40 dims before accelerating.

Figure 5.7 confirms that the belief state is robustly decodable from post-LN activations, with a plateau similar to the pre-LN results. The post-LN subspaces $\{V_k\}$ from this probing form the basis for the W_U decomposition below.

5.6.1 W_U Overlap with Probe Subspaces

For each post-LN probe subspace V_k (a 2×64 orthonormal matrix), we compute the fraction of W_U's energy that lies in that subspace:

$$\text{overlap}_k = \frac{\|V_k W_U\|_F^2}{\|W_U\|_F^2}, \tag{5.2}$$

where $\|\cdot\|_F$ denotes the Frobenius norm. By Parseval's identity, these fractions sum to 1 across all 32 subspaces. If W_U were oriented uniformly with respect to the probe subspaces, each would capture $1/32 \approx 3.1\%$ of the total energy.

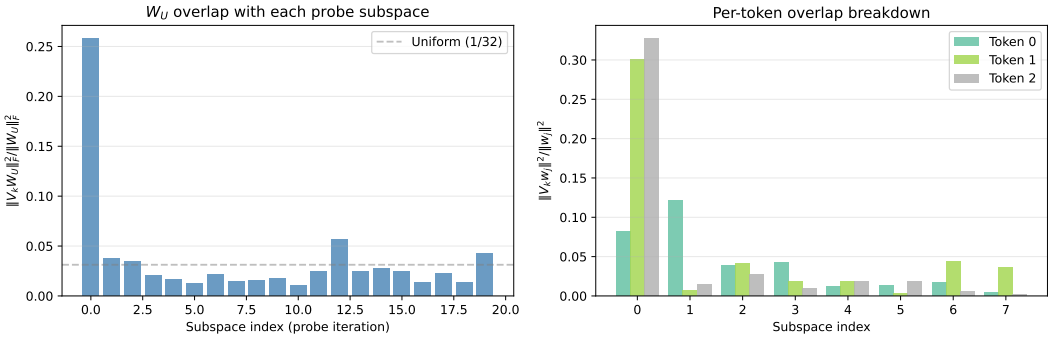


Figure 5.8: **Left:** W_U overlap fraction (Eq. 5.2) vs. probe subspace index. Subspace 0 captures 25.8% of W_U's energy—8× the uniform baseline (dashed line). **Right:** Per-token breakdown $\|V_k w_j\|^2 / \|w_j\|^2$ for the first 8 subspaces. The three token columns of W_U are read differently: token 2 is most concentrated in subspace 0 (31%), while token 1 is most concentrated in subspace 1 (30%).

Figure 5.8 shows that subspace 0—the same subspace identified as causally dominant by the residual-stream ablation—captures 25.8% of W_U's energy, far exceeding the 3.1% uniform baseline. The remaining subspaces each capture $\leq$5.7%. The per-token breakdown (right panel) reveals that different columns of W_U are read from partially different subspaces, reflecting the fact that optimal next-token probabilities depend on the belief state differently for each token.

5.6.2 Effective Emission Matrix

The Bayes-optimal predictor for an HMM outputs $P(x_t = j \mid \text{history}) = \sum_i b_i O_{ji}$, where b is the belief state and $O_{ji} = P(\text{observe } j \mid \text{state } i)$ is the observation matrix. In logit space, this corresponds to $\text{logit}_j \propto \log(\sum_i b_i O_{ji})$, which for the nearly-diagonal Mess3 observation matrix is well approximated by $\log(O^\top) \cdot b$ (up to a constant shift absorbed by softmax).

We extract the effective map from belief state to logits *through a single subspace* V_k. The construction is purely from weights:

1. The probe maps activations to belief: $\hat{b} = W_{\text{probe}}\, h + \text{intercept}$, where $W_{\text{probe}} \in \mathbb{R}^{3\times 64}$.
2. The subspace projection of h contributes logits: $\ell_k = (V_k^\top V_k h)^\top W_U$.

3. Composing via the pseudoinverse: $M_{\text{eff}} = C^\top B$, where $C = V_k \,\text{pinv}(W_{\text{probe}}) \in \mathbb{R}^{r\times 3}$ and $B = V_k\, W_U \in \mathbb{R}^{r\times 3}$.

This yields a 3×3 matrix M_{eff} that maps belief states to logit contributions through subspace k.

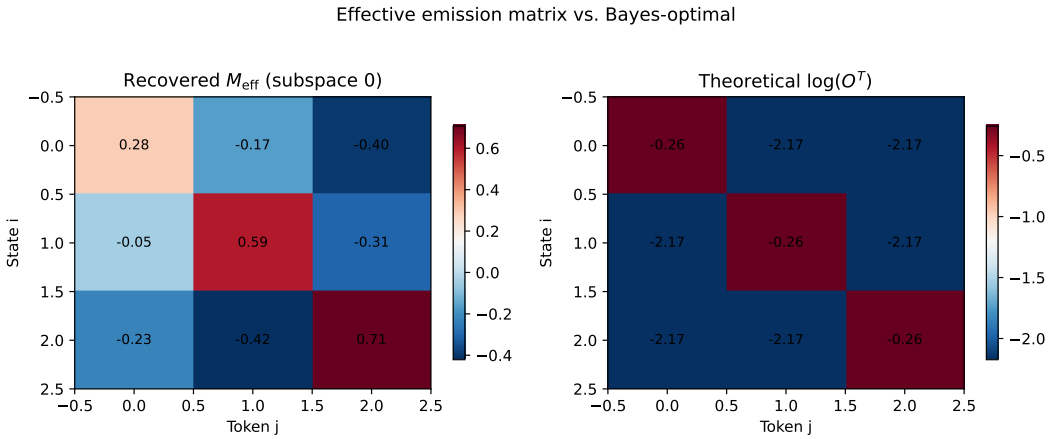


Figure 5.9: Effective emission matrix M_{eff} through subspace 0 (left) vs. theoretical $\log(O^\top)$ (right). Both are 3×3 matrices mapping (state i) $\to$ (token j). M_{eff} exhibits the same diagonal-dominant structure as $\log(O^\top)$: the highest entry in each row corresponds to the token most likely emitted by that state. The centered cosine similarity between M_{eff} and $\log(O^\top)$ is 0.928.

Figure 5.9 shows that M_{eff} through subspace 0 recovers the qualitative structure of $\log(O^\top)$: it is diagonal-dominant, with positive entries on the diagonal (state i predicts token i) and negative entries off-diagonal. The absolute scales differ because M_{eff} represents the contribution from only one subspace and is defined up to an overall scale and shift. The column-centered cosine similarity between M_{eff} and $\log(O^\top)$ is 0.928, confirming that the dominant subspace implements a close approximation to the Bayes-optimal emission structure.

Direct empirical verification. To verify that the pathway through subspace 0 produces predictions close to Bayes-optimal, we computed $\text{softmax}(M_{\text{eff}} \cdot b)$ for each sample's belief state b and compared it to the ground-truth $O^\top b$. The per-token Pearson correlations are 0.94–0.998, and the mean KL divergence $D_{\text{KL}}(O^\top b \| \text{softmax}(M_{\text{eff}} \cdot b)) = 0.108$ nats, confirming that the effective map through a single 2D subspace closely approximates the observation matrix.

5.6.3 W_U Ablation

We causally test the importance of each subspace for W_U by zeroing out its component: $W_U' = W_U - V_k^\top (V_k W_U)$. This removes W_U's ability to read from subspace k without modifying the residual stream activations.

Figure 5.10 confirms the causal dominance of subspace 0 from the W_U side. The per-subspace ablation (left) shows a clear hierarchy: subspace 0 (KL = 0.052) is an order of magnitude more important than any other individual subspace. The cumulative ablation (right) saturates after $\sim$3 subspaces (6 dimensions), which is expected: W_U has rank 3, so removing 6 orthogonal dimensions from a rank-3 matrix necessarily destroys most of its energy regardless of which directions are removed.

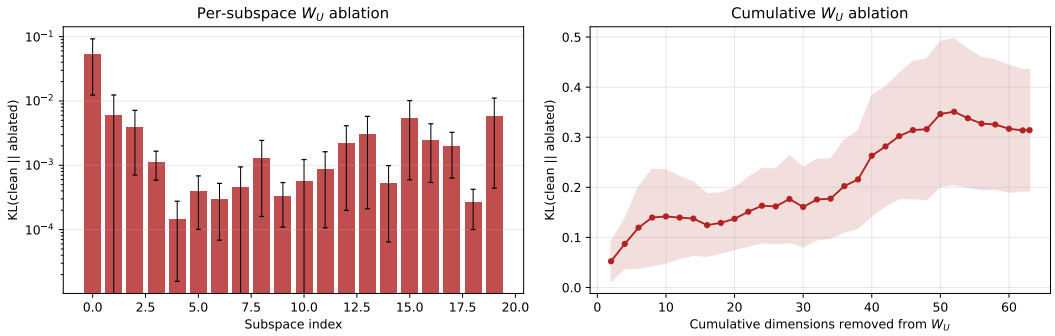


Figure 5.10: **Left:** Per-subspace W_U ablation on a log scale. Removing subspace 0 causes KL = 0.052; all other individual subspaces cause KL < 0.01. **Right:** Cumulative W_U ablation as a function of dimensions removed. The curve rises steeply in the first 2–6 dimensions (subspaces 0–2) and saturates around 0.3 nats, consistent with W_U's rank-3 constraint: once 6+ dimensions are removed, the remaining W_U projection space is severely diminished regardless of which specific subspaces are ablated.

5.6.4 Logit Contribution Analysis

Finally, we decompose the model's logits into per-subspace contributions. For each sample with post-LN activation h, the logit contribution from subspace k is:

$$\ell_k = (V_k^\top V_k\, h)^\top W_U. \tag{5.3}$$

We measure two quantities for each subspace: (i) the variance of ℓ_k across samples (how much that subspace contributes to logit *variability*), and (ii) the correlation between ℓ_k and the Bayes-optimal logits $\log(O^\top b)$.

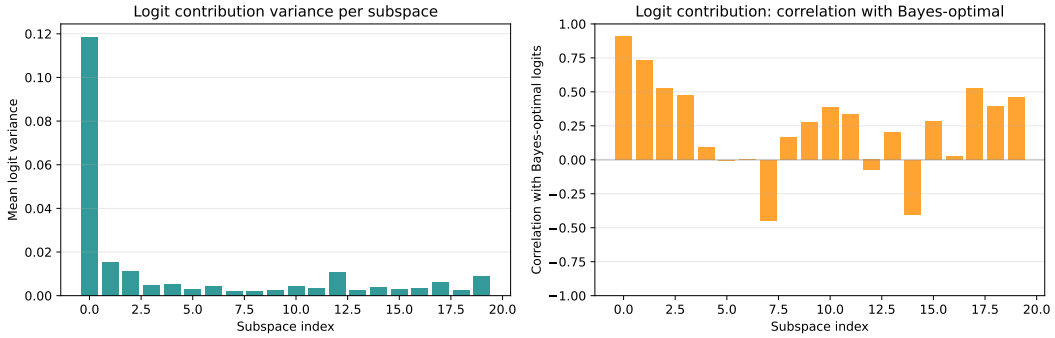


Figure 5.11: **Left:** Mean logit variance per subspace. Subspace 0 contributes 8× the variance of subspace 1, dominating the variability in the model's output. **Right:** Correlation of each subspace's logit contribution with Bayes-optimal logits. Subspace 0 achieves correlation 0.91; subsequent subspaces show weaker and often inconsistent correlations.

Figure 5.11 provides a complementary view. The variance decomposition (left) shows that subspace 0 dominates the logit signal: its contribution has 8× the variance of the next subspace. The Bayes-optimal correlation (right) confirms that subspace 0's contribution ($\rho = 0.91$) is most aligned with the theoretically optimal prediction. Later subspaces show weaker and sometimes negative correlations, consistent with their small and noisy contributions.

Summary. The unembedding analysis closes the loop from belief-state representations to next-token predictions. The picture is coherent: (i) W_U reads predominantly from subspace 0, the same subspace identified as causally dominant by residual-stream ablation; (ii) through that subspace, the model implements an effective belief-to-logit map that recovers the HMM observation matrix with centered cosine similarity 0.93; (iii) ablating W_U's access to subspace 0 causes the largest output shift, and cumulative ablation saturates after $\sim$3 subspaces due to W_U's rank-3 constraint; and (iv) subspace 0 contributes 8$\times$ more logit variance and the highest Bayes-optimal correlation of any subspace.

5.7 Discussion

5.7.1 The Nature of Redundancy

The fact that the belief state is linearly readable from over 25 independent subspaces spanning most of the 64-dimensional residual stream raises the question: is this deliberate or incidental?

In a linear model, the output logits are computed as $\text{logit} = W_U\, a + b_U$, where $W_U \in \mathbb{R}^{3\times 64}$ is the unembedding matrix and a is the final residual stream. The unembedding only "reads" the projection of a onto the row space of W_U, which is at most 3-dimensional (for a 3-token vocabulary). In a transformer with nonlinear MLP layers and attention, the situation is more complex: intermediate layers can route information through different subspaces, so multiple copies at an earlier layer may serve different downstream computations.

However, the ablation results suggest that even at the *final* layer, only the first iteration's subspace carries substantial causal weight. The remaining copies appear to be **representational byproducts**—linear structure that arises naturally from the network's computation but is not directly utilised for the output.

One plausible mechanism is that each attention head and MLP layer writes a belief-state-correlated component into the residual stream as part of its computation, and these components accumulate across layers. Since the residual stream is a sum of all layer outputs, any layer that computes a function of the belief state will leave a linearly decodable trace, even if that trace is not read by subsequent layers.

5.7.2 Layer Hierarchy and the Residual Stream

The exponential growth of causal importance across layers ($0.003 \to 0.028 \to 0.058 \to 0.112$) aligns with the standard picture of transformer computation: early layers build up representations that are progressively refined, and the final layer's output is what directly determines predictions.

The near-zero causal importance of layer 0's belief-state subspace is particularly striking. The model encodes the belief state with $R^2 = 0.997$ at layer 0, yet ablating all 20 belief-state-correlated dimensions barely affects the output. This means the subsequent layers do not read the belief state from the directions where a linear probe finds it at layer 0. They may instead reconstruct it through attention and MLP operations, or read it from entirely different (nonlinear) features of the early residual stream.

5.7.3 Decodability $\neq$ Causal Use

This experiment provides a concrete demonstration of a well-known but often underappreciated point in mechanistic interpretability: **the existence of a linear probe does not establish**

causal relevance. All 40 subspaces across 4 layers and 10 iterations have $R^2 > 0.98$, yet their KL divergences span nearly four orders of magnitude (from 10^{-5} to 10^{-2}).

The iterative probing methodology makes this dissociation quantitative. It allows us to say not just "the belief state is decodable" but "the belief state is decodable from N independent subspaces, of which only the first at layer L is causally important."

5.7.4 Comparison to Superposition

In the superposition literature, networks are known to store more features than they have dimensions by encoding features in nearly-orthogonal directions. The pattern here is different: the same 2D feature (the belief state) is stored in many exactly-orthogonal copies. This is closer to **redundancy** than superposition.

One interpretation is that this redundancy is an artefact of overparameterisation: the network has 64 dimensions but only needs $\sim$2 to represent the belief state, so the gradient-based optimisation naturally distributes belief-state-correlated structure across many directions. The ablation results suggest that the model has not learned to "clean up" this redundancy—it uses a specific 2D subspace and ignores the rest.

5.8 Implementation Notes

The implementation is split across two files:

- `src/iterative_probe.py`: Pure numpy/sklearn logic for the iterative probing algorithm. Contains the dataclasses `IterationResult` and `IterativeProbeResult`, and the functions `extract_subspace()`, `project_out_subspace()`, `iterative_probe_single_layer()`, and `iterative_probe`
- `src/run_iterative_probe.py`: CLI pipeline that orchestrates data generation, activation extraction (reusing `src/activation_extraction.py`), iterative probing (Phase 1), and counterfactual ablation (Phase 2). The ablation helpers `ablate_subspace()` and `compute_counterfactual()` are defined here.

The CLI supports `--skip-counterfactual` for running Phase 1 alone, and `--force` to re-run over existing results. Output is saved to {`model_dir`}`/iterative_probe_results/` as three files: `iterative_metrics.csv`, `counterfactual_metrics.csv`, and `subspaces.npz`.

5.9 Conclusion

Iterative probing reveals that a 4-layer, 64-dimensional transformer trained on the Mess3 HMM stores the 2D belief state across nearly all of its residual stream dimensions: R^2 remains above 0.93 through 25 successive projections (50 dims removed) before collapsing only as the space is exhausted. Despite this pervasive redundancy, counterfactual ablation—both per-subspace and cumulative-progressive—shows that only the first subspace at the final layer carries substantial causal weight for next-token prediction. The cumulative-progressive curves confirm that marginal causal importance diminishes rapidly: the first 2 dimensions at layer 3 account for the lion's share of the KL shift, with subsequent subspaces contributing only incrementally.

This dissociation between decodability and causal importance has methodological implications: probe R^2 alone is insufficient for understanding which representations a network actually relies

upon. The iterative probing framework, augmented with cumulative-progressive ablation, provides a structured way to quantify both the degree of redundancy and the causal hierarchy among redundant copies.

Bibliography

[1] S. Shai, S. Vardi, I. Arad, and N. Razin, "Transformers Represent Belief State Geometry in their Residual Stream," *arXiv preprint arXiv:2405.15943*, 2024.

[2] A. Rahimi and B. Recht, "Random Features for Large-Scale Kernel Machines," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2007.

[3] S. Shen, Z. Dong, J. Ye, L. Ma, Z. Yue, D. Bail, and Z. Liu, "Reservoir Transformers," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.